

Thiết Kế Cơ Sở Dữ Liệu

Đề Tài: LivingDocs

Nhóm 1

2026

Mục lục

Giới Thiệu Đề Tài	iii
I Thiết Kế Quan Niệm	1
1 Thuật toán 2.1 & Mô hình Conceptual ERD	3
1.1 Phân loại Tác nhân & Phạm vi Nghiệp vụ	4
1.2 Chi tiết các Thực thể Dữ liệu Cốt lõi (Core Data Entities) . .	5
1.3 Mô hình hóa Mối kết hợp & Bản số Dữ liệu	9
2 Sơ đồ Quan niệm ERD / UML Class Diagram	15
2.1 Sơ đồ ERD mức khái niệm	16
2.2 Sơ đồ Class	17
II Thiết kế Logic	19
3 Áp dụng Thuật toán 2.2 (Lược đồ Quan hệ & Chuẩn hóa) 3NF	21
3.1 Ánh xạ Thực thể, Mối kết hợp & Phẳng hóa Thuộc tính (Thuật toán 2.2)	22
3.2 Lược đồ Quan hệ Lô-gíc (Logical Schema)	25
4 Tổng kết Lược đồ Quan hệ Lô-gíc-Chuẩn hóa 3NF (Stage 2)	35
4.1 Sơ đồ ERD Mức Logic Hệ thống LivingDocs	37
III Thiết Kế Vật Lý	39
5 Thiết Kế CSDL Vật Lý & DDL SQL Theo Phân Hệ	41

5.1	Ảnh Xạ Lược Đồ ER	41
IV	Tổng Kết	63
6	Tổng Kết	65

Giới Thiệu Đề Tài

Tổng quan đề tài

Trong quy trình phát triển phần mềm hiện đại, việc duy trì tính nhất quán giữa tài liệu kỹ thuật (*Documentation*) và mã nguồn thực tế (*Git Codebase*) là một thách thức lớn. Hiện tượng "**Trôi lệch tài liệu**" (*Documentation Drift*) thường xuyên xảy ra khi mã nguồn liên tục được cập nhật nhưng tài liệu không được điều chỉnh kịp thời, dẫn đến lãng phí thời gian bảo trì và gia tăng rủi ro trong vận hành hệ thống.

Hệ thống **LivingDocs** ra đời như một giải pháp toàn diện giúp tự động hóa việc đồng bộ, kiểm duyệt và quản lý tài liệu kỹ thuật theo vết mã nguồn. Hệ thống tích hợp công nghệ AI (*AI Grounding*) để đối soát minh chứng trực tiếp giữa các đoạn mã nguồn với văn bản tài liệu, phát hiện và cảnh báo kịp thời sự trôi lệch dữ liệu.

Phạm vi Kiến trúc Cơ sở Dữ liệu

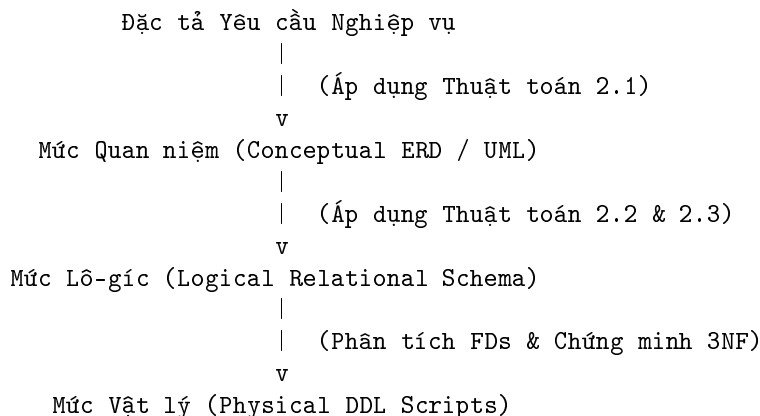
Nhóm thực hiện đề tài xây dựng và thiết kế kiến trúc Cơ sở dữ liệu (CSDL) cho hệ thống **LivingDocs**, bao phủ 5 phân hệ nghiệp vụ nòng cốt:

- **Phân hệ Quản trị & Phân quyền (*Auth & Workspace Domain*):** Quản lý thông tin người dùng (**users**), tổ chức không gian làm việc (**workspaces**), cấu trúc vai trò (**roles**) và chi tiết quyền hạn (**permissions**).
- **Phân hệ Tích hợp Mã nguồn (*Git Repository Domain*):** Đồng bộ cấu trúc dữ liệu từ Git bao gồm kho mã nguồn (**repositories**), nhánh (**branches**), lịch sử thay đổi (**commits**), yêu cầu hợp nhất (**pull_requests**) và các thực thể mã nguồn thu được từ cây AST (**code_entities**).

- **Phân hệ Tài liệu & AI Minh chứng (*Documentation & AI Domain*):** Quản lý dàn ý mẫu (`templates`), các mục dàn ý (`template_sections`), biến chèn (`placeholders`), tài liệu (`documents`), các phiên bản tài liệu (`document_versions`), dữ liệu đối soát AI (`grounding_evidences`) và hệ thống cảnh báo trôi lệch (`drift_alerts`).
- **Phân hệ Kiểm duyệt Kỹ thuật (*Code Review Domain*):** Quản lý các phiên review tài liệu (`reviews`), các quyết định phê duyệt (`approvals`) và luồng bình luận góp ý (`review_comments`).
- **Phân hệ Cấu hình & Nhật ký (*Preferences & Audit Logs*):** Quản lý cấu hình thông báo cá nhân (`user_notification_preferences`), tích hợp kết nối GitHub (`github_connections`) và lưu trữ nhật ký biến động hệ thống (`change_logs`).

Phương pháp Nghiên cứu và Quy trình Triển khai CSDL

Để đảm bảo CSDL của hệ thống LivingDocs đạt tính toàn vẹn, tối ưu hiệu năng và triệt tiêu dư thừa dữ liệu, đồ án áp dụng quy trình thiết kế CSDL chuẩn mực qua 3 mức biểu diễn kết hợp triển khai các thuật toán nền tảng của môn học Thiết kế Cơ sở Dữ liệu:



Phần I

Thiết Kế Quan Niệm

Chương 1

Thuật toán 2.1 & Mô hình Conceptual ERD

Ở mức thiết kế quan niệm (*Conceptual Design Level*), mục tiêu cốt lõi là chuyển hóa các mô tả nghiệp vụ thô, yêu cầu chức năng và quy tắc hệ thống thành mô hình dữ liệu trừu tượng, hoàn toàn độc lập với Hệ quản trị cơ sở dữ liệu (DBMS). Đồ án áp dụng **Thuật toán 2.1** (*Algorithm 2.1: Thiết kế Sơ đồ ER từ Yêu cầu Nghiệp vụ*) để đảm bảo tính toàn vẹn và phản ánh chính xác các nghiệp vụ cốt lõi của hệ thống **LivingDocs — Design and Implementation of an AI-Assisted Self-Updating Document System for Engineering Teams**.

Quá trình này tiếp nhận đầu vào (*Input*) là bài toán giải quyết hiện tượng trôi lệch tài liệu (*Documentation Drift*), bộ yêu cầu chức năng từ 5 vai trò người dùng (*Developer, Staff, Manager, Technical Lead, Administrator*) và kết xuất đầu ra (*Output*) là Sơ đồ Quan niệm (*Conceptual ERD/UML*) đã qua kiểm chứng.

Quy trình Thuật toán 2.1 (*ER Design Algorithm*):

- **Đầu vào (*Input*):** Tài liệu đặc tả nghiệp vụ LivingDocs.
- **Bước 1:** Xác định phạm vi dữ liệu cần lưu trữ (*Data Scope Definition*).
- **Bước 2:** Trích xuất cú pháp đặc tả: Liệt kê danh từ (ứng viên thực thể), động từ (ứng viên mối kết hợp), số lượng và ràng buộc.
- **Bước 3:** Lựa chọn Thực thể, Thuộc tính, Khóa chính (PK) và Mối kết hợp.

- **Bước 4:** Gán bản số (*Cardinality*), mức tham gia (*Participation*), xác định các Thực thể yếu (*Weak Entities*) nếu có.
- **Bước 5:** Kiểm thử mô hình bằng dữ liệu mẫu và các kịch bản kiểm thử biên (*Edge Cases*).
- **Đầu ra (*Output*):** Sơ đồ ERD/UML Quan niệm hoàn chỉnh đã qua kiểm chứng.

1.1 Phân loại Tác nhân & Phạm vi Nghiệp vụ

Tác nhân	Mục tiêu	Nhiệm vụ & Quyền hạn chính	Phạm vi dữ liệu	Đặc điểm cốt lõi
Developer	Khởi tạo và sử dụng tài liệu phần mềm.	Kết nối GitHub và chọn Repository vào Workspace; yêu cầu AI sinh tài liệu theo Template; xem Grounding Evidence; chỉnh sửa bản nháp và Submit; bật/tắt Auto-update; xem Version History và Drift Alert.	Workspace và Repository được cấp quyền; không có quyền Approve hoặc Publish.	Là tác nhân duy nhất khởi tạo vòng đời tài liệu, bắt đầu luồng AI → Staff → Manager .
Staff	Thực hiện review kỹ thuật cấp 1.	Xem Review Queue; kiểm tra Grounding Evidence, Source File, Code Entity, Commit, Code Diff và Change Log; kiểm tra Template; Comment, sửa lỗi nhỏ; Approve hoặc Reject và trả về Developer.	Toàn bộ bối cảnh kỹ thuật của tài liệu đang review; không có quyền Publish hoặc Commit.	Là cổng lọc kỹ thuật , phát hiện sai sót và AI hallucination trước khi chuyển đến Manager.

Tác nhân	Mục tiêu	Nhiệm vụ & Quyền hạn chính	Phạm vi dữ liệu	Đặc điểm cốt lõi
Manager	Phê duyệt cuối và quyết định xuất bản tài liệu.	Xem kết quả Staff review, Comment và Grounding Evidence; thực hiện Approve, Reject hoặc Rollback; quản lý danh mục tài liệu cần Manager approval; xem lịch sử Approval.	Tài liệu đã qua Staff review và thuộc phạm vi quản lý.	Là tác nhân duy nhất có quyền Publish hoặc Commit tài liệu vào Repository thật.
Technical Lead	Quản trị chất lượng và chính sách tài liệu.	Quản lý Documentation Collection và Ownership; thiết lập chính sách và ngưỡng phát hiện tài liệu lỗi thời ; quản lý Template, Section, Heading, Placeholder và ánh xạ Placeholder ↔ Code Entity.	Toàn bộ Workspace và Repository thuộc phạm vi quản lý.	Tập trung vào Policy, Threshold và Template , không review từng tài liệu như Manager.
Admin	Vận hành hạ tầng và quản trị hệ thống.	Quản lý Tài khoản & Phân quyền: User, Role, Permission, Workspace và Repository và cấu hình tích hợp & hạ tầng như GitHub App, OAuth, GitHub Actions, tích hợp GitLab, Jira, Slack; cấu hình ngôn ngữ và AI Model.	Toàn bộ hệ thống ở cấp độ hạ tầng và kỹ thuật; không review hoặc approve tài liệu.	Là tác nhân duy nhất quản lý cấu hình AI Model như LLM và Embedding, nằm ngoài luồng phê duyệt tài liệu.

1.2 Chi tiết các Thực thể Dữ liệu Cốt lõi (Core Data Entities)

Tên thực thể	Loại thực thể	Mô tả	Thuộc tính
User	Thực thể mạnh	Người dùng cuối trong hệ thống (Developer, Staff, Manager, Tech Lead, Admin).	PK: user_id; email; password_hash; FullName (Phức hợp: FirstName, LastName); NotificationPreferences (đa trị); status; created_at; updated_at; deleted_at
Workspace	Thực thể mạnh	Không gian làm việc chứa repository và tài liệu của một nhóm/dự án.	PK: workspace_id; name; description; status; created_at; updated_at; deleted_at
Role	Thực thể mạnh	Vai trò của người dùng, được scoped theo Workspace.	PK: role_id; role_name; description; created_at; updated_at
Permission	Thực thể mạnh	Quyền hạn chi tiết trên hệ thống, ví dụ DOC_APPROVE, TEMPLATE_EDIT.	PK: permission_id; permission_code; description; created_at
GitHubConnection	Thực thể phụ thuộc User	Lưu thông tin xác thực OAuth của User với GitHub.	PK: connection_id; github_user_id; github_username; access_token; scopes (đa trị); created_at; updated_at
Repository	Thực thể mạnh	Kho chứa mã nguồn được đồng bộ vào hệ thống.	PK: repository_id; name; url; default_branch; sync_status; last_synced_at; created_at; updated_at
Branch	Thực thể yếu – phụ thuộc Repository	Nhánh phát triển trong repository.	PK: branch_id; branch_name; is_default; created_at; updated_at

Tên thực thể	Loại thực thể	Mô tả	Thuộc tính
Commit	Thực thể yếu – phụ thuộc Branch/Repository	Bản ghi một lần commit mã nguồn.	PK: commit_id; AuthorInfo (AuthorName, AuthorEmail); message; committed_at; created_at
PullRequest	Thực thể yếu – phụ thuộc Repository	Yêu cầu hợp nhất nhánh (PR) từ GitHub.	PK: pr_id; pr_number; title; description; source_branch; target_branch; status; labels (đa trị); created_at; updated_at; merged_at
CodeDiff	Thực thể yếu – phụ thuộc Commit/PR	Chi tiết thay đổi mã nguồn trong commit hoặc PR.	PK: diff_id; file_path; change_type; additions; deletions; diff_content; created_at
File	Thực thể yếu – phụ thuộc Repository	Cấu trúc tập tin mã nguồn trong kho chứa.	PK: file_id; file_path; file_type; status; created_at; updated_at
CodeEntity	Thực thể yếu – phụ thuộc File	Đơn vị mã nguồn cụ thể thu được từ AST (Class, Method, Function...).	PK: entity_id; entity_name; entity_type; signature; doc_comment; position (start_line, end_line); created_at; updated_at
Template	Thực thể mạnh	Khuôn mẫu chuẩn để AI/User sinh tài liệu.	PK: template_id; name; description; content_structure; creator_type; status; created_at; updated_at
TemplateSection	Thực thể yếu – phụ thuộc Template	Mục con/heading trong cấu trúc dàn ý của Template.	PK: section_id; heading_title; order_index; prompt_instruction; created_at; updated_at

Tên thực thể	Loại thực thể	Mô tả	Thuộc tính
Placeholder	Thực thể yếu – phụ thuộc TemplateSection	Biến/điểm chèn trong Section, nhận dữ liệu từ code.	PK: placeholder_id; variable_name; expected_data_type; description; is_required; created_at; updated_at
Document	Thực thể mạnh	Tài liệu gốc như README, API Reference, Architecture Guide, ADR...	PK: document_id; title; document_type; status; created_at; updated_at; deleted_at
DocumentVersion	Thực thể yếu – phụ thuộc Document	Bản ghi nội dung tài liệu theo từng mốc thời gian và trạng thái (Draft, In Review, Approved, Published).	PK: version_id; version_number; content; change_summary; status; ticket_codes; created_at; updated_at
ChangeLog	Thực thể yếu – phụ thuộc DocumentVersion	Lịch sử vết thay đổi chi tiết giữa các phiên bản tài liệu.	PK: log_id; change_type; change_description; diff_content; created_at; updated_at
Review	Thực thể yếu – phụ thuộc DocumentVersion	Tiến trình Staff thực hiện đánh giá kỹ thuật trên một DocumentVersion.	PK: review_id; status; decision; summary_notes; created_at; updated_at
ReviewComment	Thực thể yếu – phụ thuộc Review	Phản hồi, góp ý chi tiết trong quá trình kiểm duyệt.	PK: comment_id; content; is_resolved; created_at; updated_at
GroundingEvidence	Thực thể yếu – phụ thuộc DocumentVersion	Bảng minh chứng chứa liên kết nguồn gốc (File, CodeEntity, Commit) AI dùng để tạo nội dung, phục vụ chống hallucination.	PK: evidence_id; snippet_content; purpose; created_at

Tên thực thể	Loại thực thể	Mô tả	Thuộc tính
DriftAlert	Thực thể sự kiện/thực thể yếu	Cảnh báo phát hiện tài liệu bị lệch ngữ nghĩa/lỗi thời so với code mới.	PK: alert_id; drift_type; status; detected_at
Approval	Thực thể mối kết hợp / Sự kiện	Quyết định phê duyệt xuất bản hoặc commit tài liệu chính thức từ phía Manager.	PK: approval_id; FK: version_id; FK: user_id; decision; rejection_reason; approved_at

1.3 Mô hình hóa Mối kết hợp & Bản số Dữ liệu

Mối quan hệ & Phân loại thực thể	Bản số & Mức tham gia	Thuộc tính mối quan hệ	Mô tả nghiệp vụ
User — Workspace — Role (<participates>) <i>Mối kết hợp 3 ngôi:</i> User (Mạnh), Workspace (Mạnh), Role (Mạnh)	Tỷ lệ: M:N:1 Bản số: User (0..*), Workspace (1..*), Role (0..*) Tham gia: User (Partial 0), Workspace (Total 1), Role (Partial 0)	joined_at, status	Ghi nhận sự tham gia của Người dùng vào Workspace. Trong từng Workspace, mỗi user bắt buộc gắn đúng một Role (Scoped RBAC). Một user có thể thuộc nhiều Workspace với vai trò độc lập.
Role — Permission (<contains>) Role (Mạnh) ↔ Permission (Mạnh)	Tỷ lệ: M:N Bản số: Role (0..*) ↔ Permission (0..*) Tham gia: Role (Partial) ↔ Permission (Partial)	granted_at	Một vai trò chứa tập hợp nhiều quyền; một quyền hạn có thể thuộc về nhiều vai trò khác nhau.
User — Github Connection (<connects>) User (Mạnh) ↔ Github Connection (Phụ thuộc User)	Tỷ lệ: 1:N Bản số: User (0..*), Github Connection (1..1) Tham gia: User (Partial), Github Connection (Total)	Không có	Ghi nhận việc User ủy quyền tài khoản GitHub qua OAuth2, lưu access_token và scopes để đồng bộ repo, commit, PR, webhook.

Mối quan hệ & Phân loại thực thể	Bản số & Mức tham gia	Thuộc tính mối quan hệ	Mô tả nghiệp vụ
Workspace — Repository (<owns>) Workspace (Mạnh) ↔ Repository (Mạnh)	Tỷ lệ: 1:N Bản số: Workspace (1..1) ↔ Repository (0..*) Tham gia: Workspace (Partial), Repository (Total)	Không có	Một Workspace quản lý nhiều Repository; mỗi Repository thuộc quyền sở hữu của một Workspace.
Repository — Branch (<has_branch>) Repository (Mạnh) ↔ Branch (Yếu - Phụ thuộc Repo)	Tỷ lệ: 1:N Bản số: Repository (1..1) ↔ Branch (1..*) Tham gia: Repository (Total), Branch (Total)	Không có	Một kho chứa mã nguồn có ít nhất một nhánh mặc định; mỗi nhánh bắt buộc nằm trong duy nhất một kho chứa.
Branch — Commit (<contains_commit>) Branch (Yếu) ↔ Commit (Yếu - Phụ thuộc Branch/Repo)	Tỷ lệ: 1:N Bản số: Branch (0..*) ↔ Commit (1..1) Tham gia: Branch (Partial), Commit (Total)	Không có	Một nhánh chứa nhiều lượt commit; mỗi commit thuộc về đúng một nhánh phát triển.
User — Commit (<makes_commit>) User (Mạnh) ↔ Commit (Yếu)	Tỷ lệ: 1:N Bản số: User (0..*) ↔ Commit (1..1) Tham gia: User (Partial), Commit (Total)	Không có	Một người dùng thực hiện nhiều commit; mỗi commit được tạo bởi đúng một người dùng.
Repository — PullRequest (<receives_pr>) Repository (Mạnh) ↔ PullRequest (Yếu - Phụ thuộc Repo)	Tỷ lệ: 1:N Bản số: Repository (0..*) ↔ PullRequest (1..1) Tham gia: Repository (Partial), PullRequest (Total)	Không có	Một Repository chứa nhiều Yêu cầu hợp nhất (PR); mỗi PR gửi về đúng một kho chứa.
Commit — Code Diff (<generates_diff>) Commit (Yếu) ↔ CodeDiff (Yếu - Phụ thuộc Commit/PR)	Tỷ lệ: 1:N Bản số: Commit (1..*) ↔ CodeDiff (1..1) Tham gia: Commit (Total), CodeDiff (Total)	Không có	Một Commit sinh ra một hoặc nhiều bản ghi khác biệt code (Diff); mỗi bản ghi diff thuộc về duy nhất một commit.

Mối quan hệ & Phân loại thực thể	Bản số & Mức tham gia	Thuộc tính mối quan hệ	Mô tả nghiệp vụ
PullRequest — Commit (<code><includes_commit></code>) PullRequest (Yếu) ↔ Commit (Yếu)	Tỷ lệ: 1:N Bản số: PullRequest (1..*) ↔ Commit (1..1) Tham gia: PullRequest (Total), Commit (Partial)	Không có	Một Yêu cầu hợp nhất (PR) tổng hợp danh sách gồm một hoặc nhiều commit; mỗi commit thuộc về đúng một PR.
Commit — File (<code><modifies></code>) Commit (Yếu) ↔ File (Yếu - Phụ thuộc Repo)	Tỷ lệ: M:N Bản số: Commit (1..*) ↔ File (0..*) Tham gia: Commit (Total), File (Partial)	<code>change_type</code> , <code>lines_added</code> , <code>lines_deleted</code>	Một Commit tác động chỉnh sửa ít nhất một File; một File có thể chịu sự biến đổi qua nhiều lượt Commit.
Repository — File (<code><contains_file></code>) Repository (Mạnh) ↔ File (Yếu - Phụ thuộc Repo)	Tỷ lệ: 1:N Bản số: Repository (0..*) ↔ File (1..1) Tham gia: Repository (Partial), File (Total)	Không có	Một Repository quản lý cấu trúc cây thư mục chứa nhiều File mã nguồn; mỗi File thuộc một kho chứa duy nhất.
File — Code Entity (<code><parses_into></code>) File (Yếu) ↔ CodeEntity (Yếu - Phụ thuộc File)	Tỷ lệ: 1:N Bản số: File (0..*) ↔ CodeEntity (1..1) Tham gia: File (Partial), CodeEntity (Total)	Không có	Một File code qua bóc tách AST thu được nhiều thực thể code (Class, Function); mỗi thực thể code nằm trọn trong đúng một File.
CodeEntity — CodeEntity (<code><depends_on></code>) <i>Đệ quy:</i> CodeEntity (Yếu) ↔ CodeEntity (Yếu)	Tỷ lệ: M:N Bản số: Phụ thuộc (0..*), Được phụ thuộc (0..*) Tham gia: Cả 2 phía Partial	<code>dependency_type</code>	Xác định mối quan hệ phụ thuộc (gọi hàm, kế thừa, import) giữa các đơn vị code trong hệ thống.
Workspace — Template (<code><defines></code>) Workspace (Mạnh) ↔ Template (Mạnh)	Tỷ lệ: 1:N Bản số: Workspace (0..*) ↔ Template (1..1) Tham gia: Workspace (Partial), Template (Total)	Không có	Một Workspace định nghĩa nhiều khuôn mẫu tài liệu; mỗi mẫu tài liệu thuộc sở hữu của một Workspace.

Mối quan hệ & Phân loại thực thể	Bản số & Mức tham gia	Thuộc tính mối quan hệ	Mô tả nghiệp vụ
Template — TemplateSection (<code><has_section></code>) Template (Mạnh) ↔ TemplateSection (Yếu - Phụ thuộc Template)	Tỷ lệ: 1:N Bản số: Template (1..*) ↔ TemplateSection (1..1) Tham gia: Template (Total), TemplateSection (Total)	Không có	Một Template cấu thành từ ít nhất một Section dài ý; mỗi Section nằm trên đúng một Template.
TemplateSection — Placeholder (<code><contains_placeholder></code>) TemplateSection (Yếu) ↔ Placeholder (Yếu - Phụ thuộc Section)	Tỷ lệ: 1:N Bản số: TemplateSection (0..*) ↔ Placeholder (1..1) Tham gia: TemplateSection (Partial), Placeholder (Total)	Không có	Mỗi mục trong mẫu tài liệu chứa nhiều biến chèn tự động. Khi xóa Section, các Placeholder nằm trong đó bị xóa theo.
Placeholder — CodeEntity (<code><maps_to></code>) Placeholder (Yếu) ↔ CodeEntity (Yếu)	Tỷ lệ: M:N Bản số: Placeholder (0..*) ↔ CodeEntity (0..*) Tham gia: Placeholder (Partial), CodeEntity (Partial)	<code>extraction_rule</code> , <code>is_active</code>	Ảnh xạ chỉ thị cho AI biết lấy thông tin từ thành phần code nào để điền vào biến trong mẫu.
Workspace — Document (<code><manages></code>) Workspace (Mạnh) ↔ Document (Mạnh)	Tỷ lệ: 1:N Bản số: Workspace (0..*) ↔ Document (1..1) Tham gia: Workspace (Partial), Document (Total)	Không có	Workspace quản lý danh mục tài liệu gốc của dự án; một tài liệu thuộc về đúng một Workspace.
User — Document (<code><authors></code>) User (Mạnh) ↔ Document (Mạnh)	Tỷ lệ: 1:N Bản số: User (0..*) ↔ Document (1..1) Tham gia: User (Partial), Document (Total)	Không có	Người dùng đứng tên tác giả khởi tạo tài liệu; mỗi tài liệu có một tác giả duy nhất.

Mối quan hệ & Phân loại thực thể	Bản số & Mức tham gia	Thuộc tính mối quan hệ	Mô tả nghiệp vụ
Document — Document Version (<has_version>) Document (Mạnh) ↔ Document Version (Yếu - Phụ thuộc Document)	Tỷ lệ: 1:N Bản số: Document (1..*) ↔ Document Version (1..1) Tham gia: Document (Total), Document Version (Total)	Không có	Một tài liệu chính thức chứa ít nhất một phiên bản nội dung; mỗi phiên bản đại diện cho một mốc chỉnh sửa.
Document Version — ChangeLog (<tracks_change>) Document Version (Yếu) ↔ ChangeLog (Yếu - Phụ thuộc DocVersion)	Tỷ lệ: 1:N Bản số: Document Version (0..*) ↔ ChangeLog (1..1) Tham gia: Document Version (Partial), ChangeLog (Total)	Không có	Mỗi phiên bản tài liệu có thể ghi nhận nhiều lượt nhật ký biến động nội dung chi tiết.
Document Version — Review (<has_review>) Document Version (Yếu) ↔ Review (Yếu - Phụ thuộc DocVersion)	Tỷ lệ: 1:N Bản số: Document Version (0..*) ↔ Review (1..1) Tham gia: Document Version (Partial), Review (Total)	Không có	Một phiên bản tài liệu có thể nhận 0 hoặc nhiều lượt đánh giá kỹ thuật; mỗi lượt đánh giá thuộc về duy nhất một phiên bản.
User — Review (<performs_review>) User (Mạnh) ↔ Review (Yếu)	Tỷ lệ: 1:N Bản số: User (0..*) ↔ Review (1..1) Tham gia: User (Partial), Review (Total)	Không có	Một người dùng có thể thực hiện 0 hoặc nhiều lượt đánh giá; mỗi lượt review do đúng một người dùng chịu trách nhiệm.
Review — ReviewComment (<contains_comment>) Review (Sự kiện/Yếu) ↔ ReviewComment (Yếu - Phụ thuộc Review)	Tỷ lệ: 1:N Bản số: Review (0..*) ↔ Review Comment (1..1) Tham gia: Review (Partial), Review Comment (Total)	Không có	Một phiên kiểm duyệt kỹ thuật chứa nhiều bình luận góp ý chi tiết; mỗi bình luận gắn chặt với một lượt review.

Mối quan hệ & Phân loại thực thể	Bản số & Mức tham gia	Thuộc tính mối quan hệ	Mô tả nghiệp vụ
Document Version — Approval (<code><requires_approval></code>) DocumentVersion (Yếu) ↔ Approval (Yếu - Phụ thuộc DocVersion)	Tỷ lệ: 1:N Bản số: DocumentVersion (0..*) ↔ Approval (1..1) Tham gia: DocumentVersion (Partial), Approval (Total)	Không có	Một phiên bản tài liệu có thể có 0 hoặc nhiều lượt phê duyệt; mỗi lượt phê duyệt bắt buộc thuộc về đúng một phiên bản.
User — Approval (<code><grants_approval></code>) User (Mạnh) ↔ Approval (Yếu)	Tỷ lệ: 1:N Bản số: User (0..*) ↔ Approval (1..1) Tham gia: User (Partial), Approval (Total)	Không có	Một Manager có thể thực hiện 0 hoặc nhiều đợt phê duyệt; mỗi bản ghi phê duyệt do đúng một Manager ra quyết định.
Document Version — Grounding Evidence (<code><has_evidence></code>) DocumentVersion (Yếu) ↔ GroundingEvidence (Yếu - Phụ thuộc DocVersion)	Tỷ lệ: 1:N Bản số: DocumentVersion (0..*) ↔ GroundingEvidence (1..1) Tham gia: DocumentVersion (Partial), GroundingEvidence (Total)	Không có	Phiên bản tài liệu lưu trữ bằng minh chứng đối soát nguồn gốc code để chống ảo giác (hallucination) của AI.
Grounding Evidence — CodeEntity (<code><references_code></code>) GroundingEvidence (Yếu) ↔ CodeEntity (Yếu)	Tỷ lệ: M:N Bản số: GroundingEvidence (0..*) ↔ CodeEntity (0..*) Tham gia: GroundingEvidence (Partial), CodeEntity (Partial)	<code>similarity_score</code> , <code>match_type</code>	Minh chứng AI liên kết trực tiếp với các đơn vị code cụ thể trong hệ thống để phục vụ đối soát sai lệch.
Grounding Evidence — DriftAlert (<code><detects></code>) GroundingEvidence (Yếu) ↔ DriftAlert (Thực thể sự kiện)	Tỷ lệ: 1:N Bản số: GroundingEvidence (0..*) ↔ DriftAlert (1..1) Tham gia: GroundingEvidence (Partial), DriftAlert (Total)	Không có	Minh chứng đối soát khi phát hiện sự bất đồng giữa tài liệu và mã nguồn mới sẽ kích hoạt cảnh báo sai lệch (Drift Alert).

Chương 2

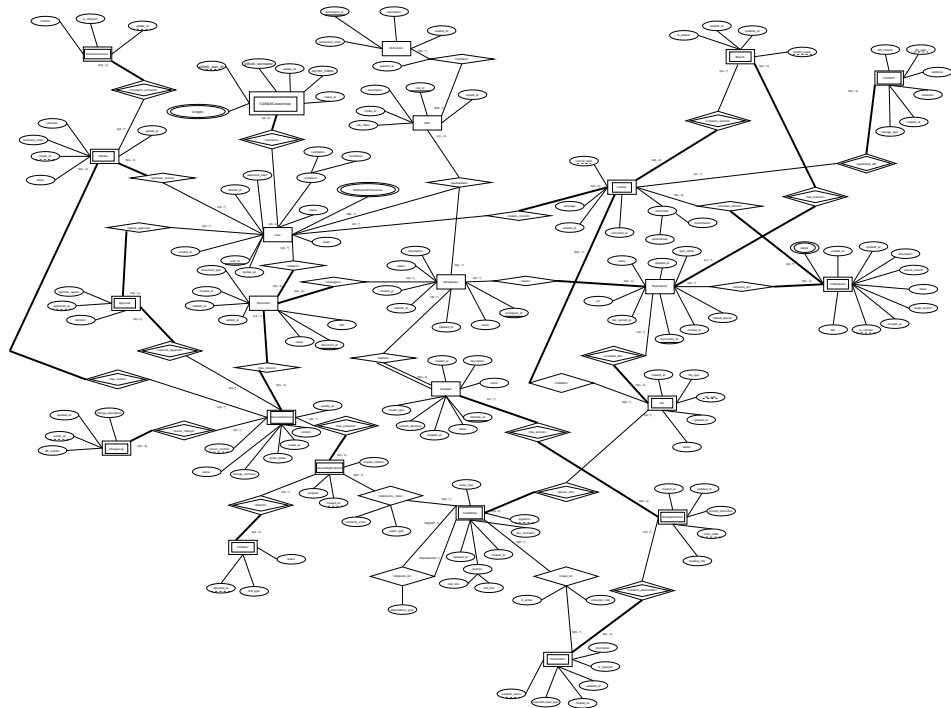
Sơ đồ Quan niệm ERD / UML Class Diagram

Đầu ra của Stage 1 (Thiết kế quan niệm) bao gồm sơ đồ ERD và UML Class Diagram, biểu diễn trừu tượng toàn bộ các thực thể, thuộc tính và mối kết hợp nghiệp vụ của hệ thống LivingDocs ở mức logic cao nhất, hoàn toàn độc lập với bất kỳ hệ quản trị CSDL vật lý nào.

Mô hình đã xác định chính xác các bản số ($1 : 1$, $1 : N$, $M : N$) và quy tắc ràng buộc miền, bao phủ trọn vẹn từ quản trị không gian làm việc, cấu trúc kho code cho đến luồng kiểm duyệt 2 cấp ($\text{Staff} \rightarrow \text{Manager}$) và cơ chế lưu minh chứng AI chống ảo giác.

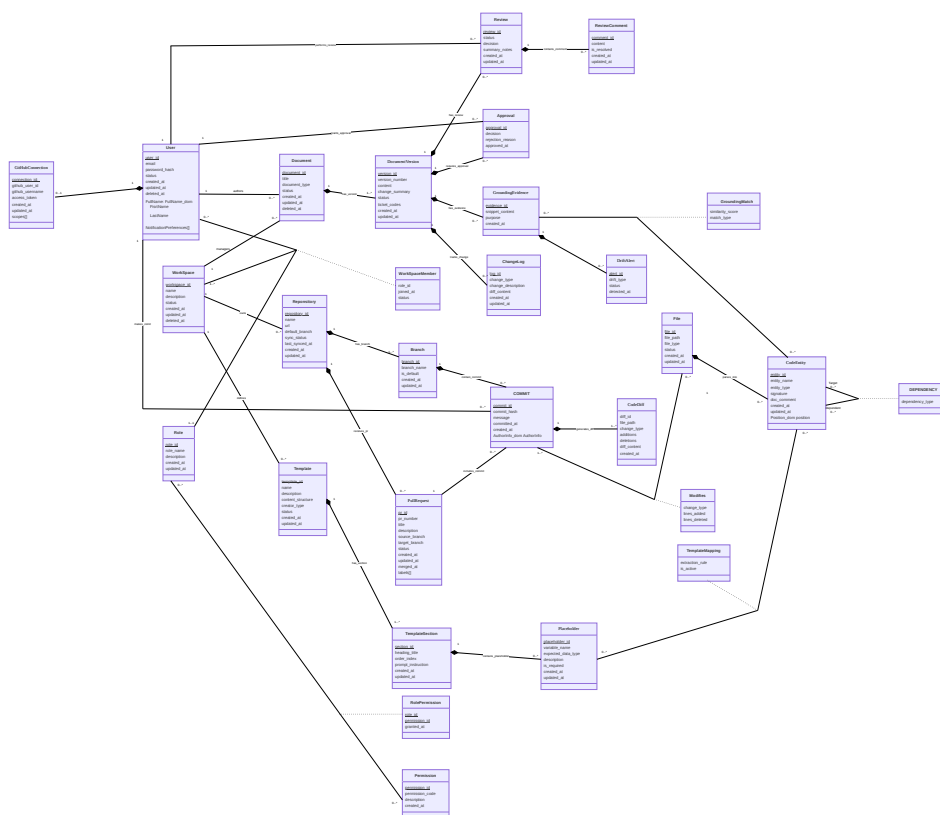
Đây là bức tranh kiến trúc dữ liệu hoàn chỉnh, đóng vai trò là đầu vào (input) chuẩn xác để thực hiện ánh xạ sang Lược đồ quan hệ (Logical Schema) và chuẩn hóa 3NF ở Stage 2.

2.1 Sơ đồ ERD mức khái niệm



Hình 2.1: Sơ đồ ERD mức khái niệm

2.2 Sơ đồ Class



Hình 2.2: Sơ đồ Class

Phần II

Thiết kế Logic

Chương 3

Áp dụng Thuật toán 2.2 (Lược đồ Quan hệ & Chuẩn hóa) 3NF

Ở mức thiết kế logic (*Logical Design Level*), mục tiêu cốt lõi là chuyển hóa mô hình dữ liệu quan niệm trừu tượng (ERD/UML) thành lược đồ quan hệ lô-gíc (*Logical Relational Schema*) cụ thể, xác định rõ cấu trúc các bảng, cột, khóa chính (PK), khóa ngoại (FK) và các ràng buộc toàn vẹn [1, 2]. Đồ án áp dụng **Thuật toán 2.2** (*Algorithm 2.2: Ánh xạ ERD sang Lược đồ Quan hệ*) để đảm bảo tính toàn vẹn tham chiếu, loại bỏ các dị thường dữ liệu và đưa toàn bộ lược đồ đạt tiêu chuẩn chuẩn hóa 3NF cho hệ thống **LivingDocs — Design and Implementation of an AI-Assisted Self-Updating Document System for Engineering Teams** [3-5].

Quá trình này tiếp nhận đầu vào (*Input*) là Sơ đồ Quan niệm (*Conceptual ERD/UML*) đã qua kiểm chứng ở Stage 1, và kết xuất đầu ra (*Output*) là Lược đồ Quan hệ Lô-gíc (*Logical Schema*) hoàn chỉnh, hồ sơ Ràng buộc Khóa ngoại/Giá trị miền (*FK & CHECK Rules*) và Báo cáo chứng minh Chuẩn hóa 3NF [3, 5-12].

Quy trình Thuật toán 2.2 (*Relational Mapping Algorithm*):

- **Đầu vào (*Input*):** Sơ đồ Quan niệm ERD/UML và từ điển dữ liệu hoàn chỉnh từ Stage 1 .
- **Bước 1 (Thực thể mạnh):** Ánh xạ các thực thể mạnh thành các bảng chính, chuyển thuộc tính đơn thành cột và thiết lập Khóa chính (PK) .

- **Bước 2 (Thực thể yếu):** Ánh xạ các thực thể yếu thành các bảng phụ thuộc, gán Surrogate PK kiểu UUID và cài đặt FK trở về thực thể chủ .
- **Bước 3 & 4 (Mối kết hợp 1 : 1 và 1 : N):** Chèn Khóa ngoại (FK) từ bảng phía 1 sang bảng phía N (hoặc phía tham gia bắt buộc) kèm các ràng buộc NOT NULL/UNIQUE phù hợp .
- **Bước 5 (Mối kết hợp $M : N$):** Khởi tạo các bảng trung gian (*Junction Tables*) chứa cặp Khóa ngoại (FK) trở về hai bảng gốc và các thuộc tính riêng của mối kết hợp .
- **Bước 6 (Thuộc tính đặc biệt):** Phẳng hóa các thuộc tính phức hợp thành các cột đơn độc lập và tách các thuộc tính đa trị thành các bảng phụ riêng .
- **Bước 7 (Ràng buộc & Chuẩn hóa):** Khai báo tập Phụ thuộc hàm (FDs), chứng minh lược đồ đạt chuẩn 3NF và thiết lập các ràng buộc toàn vẹn (*FK Constraints*) .
- **Đầu ra (*Output*):** Lược đồ Quan hệ Lô-gíc (*Logical Schema*) hoàn chỉnh đạt chuẩn 3NF và hồ sơ ràng buộc toàn vẹn .

3.1 Ánh xạ Thực thể, Mối kết hợp & Phẳng hóa Thuộc tính (Thuật toán 2.2)

1. Chuyển đổi Thực thể mạnh thành Bảng chính

STT	Thực thể (ERD)	Bảng CSDL (snake_case)	Ghi chú thiết kế
1	User	users	Khóa chính (PK): user_id (UUID)
2	Workspace	workspaces	Khóa chính (PK): workspace_id (UUID)
3	Role	roles	Khóa chính (PK): role_id (UUID)
4	Permission	permissions	Khóa chính (PK): permission_id (UUID)
5	Repository	repositories	Khóa chính (PK): repository_id (UUID)

STT	Thực thể (ERD)	Bảng CSDL (snake_case)	Ghi chú thiết kế
6	Template	templates	Khóa chính (PK): template_id (UUID)
7	Document	documents	Khóa chính (PK): document_id (UUID)

2. Chuyển đổi Thực thể yếu thành Bảng phụ

STT	Bảng phụ (Thực thể yếu)	Bảng cha trực tiếp	Khóa ngoại (FK) & Ràng buộc bổ sung
1	github_connections	users	user_id (FK, UNIQUE)
2	branches	repositories	repository_id (FK)
3	commits	branches	branch_id (FK); FK bổ sung: author_user_id → users, pr_id → pull_requests
4	pull_requests	repositories	repository_id (FK)
5	code_diffs	commits	commit_id (FK)
6	files	repositories	repository_id (FK)
7	code_entities	files	file_id (FK)
8	template_sections	templates	template_id (FK)
9	placeholders	template_sections	section_id (FK)
10	document_versions	documents	document_id (FK)
11	change_logs	document_versions	version_id (FK)
12	reviews	document_versions	version_id (FK); FK bổ sung: reviewer_user_id → users
13	review_comments	reviews	review_id (FK)
14	approvals	document_versions	version_id (FK); FK bổ sung: approver_user_id → users
15	grounding_evidences	document_versions	version_id (FK)
16	drift_alerts	grounding_evidences	evidence_id (FK)

3. Ánh xạ Quan hệ 1:1 và 1:N (Chèn Khóa ngoại FK)

STT	Bảng chứa FK (Phía N/Phụ)	Cột FK được chèn	Bảng đích (Phía 1/Chính)	Loại quan hệ
1	github_connections	user_id	users	1:1 (UNIQUE)
2	repositories	workspace_id	workspaces	1:N
3	branches	repository_id	repositories	1:N
4	commits	branch_id	branches	1:N

STT	Bảng chứa FK (Phía N/Phụ)	Cột FK được chèn	Bảng đích (Phía 1/Chính)	Loại quan hệ
5	commits	author_user_id	users	1:N
6	commits	pr_id	pull_requests	1:N (Nullable)
7	pull_requests	repository_id	repositories	1:N
8	code_diffs	commit_id	commits	1:N
9	files	repository_id	repositories	1:N
10	code_entities	file_id	files	1:N
11	templates	workspace_id	workspaces	1:N
12	template_sections	template_id	templates	1:N
13	placeholders	section_id	template_sections	1:N
14	documents	workspace_id	workspaces	1:N
15	documents	author_user_id	users	1:N
16	document_versions	document_id	documents	1:N
17	change_logs	version_id	document_versions	1:N
18	reviews	version_id	document_versions	1:N
19	reviews	reviewer_user_id	users	1:N
20	review_comments	review_id	reviews	1:N
21	approvals	version_id	document_versions	1:N
22	approvals	approver_user_id	users	1:N
23	grounding_evidences	version_id	document_versions	1:N
24	drift_alerts	evidence_id	grounding_evidences	1:N

4. Chuyển đổi Mỗi kết hợp $M : N$ thành Bảng trung gian

STT	Bảng trung gian	Mỗi kết hợp gốc	Danh sách các Khóa ngoại (FK)	Ghi chú
1	workspace_members	<i>participates</i>	user_id, workspace_id, role_id	Quan hệ 3 ngôi ($M : N : 1$)
2	role_permissions	<i>contains</i>	role_id, permission_id	Quan hệ $M : N$
3	commit_file	<i>modifies</i>	commit_id, file_id	Quan hệ $M : N$
4	dependencies	<i>depends_on</i>	dependent_entity_id, target_entity_id	Đệ quy trở về code_entities
5	template_mappings	<i>maps_to</i>	placeholder_id, code_entity_id	Quan hệ $M : N$
6	grounding_evidences_code_entities	<i>references_code</i>	evidence_id, code_entity_id	Quan hệ $M : N$

5. Xử lý Thuộc tính đặc biệt (Phức hợp & Đa trị)

STT	Loại thuộc tính	Thuộc tính gốc (ERD)	Giải pháp chuyển đổi	Cột / Bảng kết quả
1	Phức hợp	User.FullName	Phẳng hóa thành các cột	first_name, last_name trong users
2	Phức hợp	Commit.AuthorInfo	Phẳng hóa thành các cột	author_name, author_email trong commits
3	Phức hợp	CodeEntity.position	Phẳng hóa thành các cột	start_line, end_line trong code_entities
4	Đa trị	User.Notification Preferences	Tách thành bảng phụ chứa FK	Bảng user_notification_preferences
5	Đa trị	GitHubConnection.scopes	Tách thành bảng phụ chứa FK	Bảng github_connection_scopes
6	Đa trị	PullRequest.labels	Tách thành bảng phụ chứa FK	Bảng pull_request_labels

3.2 Lược đồ Quan hệ Lô-gíc (Logical Schema)

3.2.1 Phân hệ Quản trị & Xác thực

1. Bảng users (Bảng chính - Thực thể mạnh: User)

Tên Cột	Kiểu Dữ Liệu	Khóa	Ràng Buộc
user_id	UUID	PK	NOT NULL
email	-	Unique	NOT NULL
password_hash	-	-	NULL
first_name	-	-	NULL
last_name	-	-	NULL
status	-	-	NOT NULL
created_at	-	-	NOT NULL
updated_at	-	-	NOT NULL
deleted_at	-	-	NULL

2. Bảng workspaces (Bảng chính - Thực thể mạnh: Workspace)

Tên Cột	Kiểu Dữ Liệu	Khóa	Ràng Buộc
workspace_id	UUID	PK	NOT NULL
name	-	-	NOT NULL
description	-	-	NULL
status	-	-	NOT NULL
created_at	-	-	NOT NULL
updated_at	-	-	NOT NULL
deleted_at	-	-	NULL

3. Bảng roles (Bảng chính - Thực thể mạnh: Role)

Tên Cột	Kiểu Dữ Liệu	Khóa	Ràng Buộc
role_id	UUID	PK	NOT NULL
role_name	-	-	NOT NULL
description	-	-	NULL
created_at	-	-	NOT NULL
updated_at	-	-	NOT NULL

4. Bảng permissions (Bảng chính - Thực thể mạnh: Permission)

Tên Cột	Kiểu Dữ Liệu	Khóa	Ràng Buộc
permission_id	UUID	PK	NOT NULL
permission_code	-	Unique	NOT NULL
description	-	-	NULL
created_at	-	-	NOT NULL

5. Bảng github_connections (Bảng phụ - Thực thể yếu phụ thuộc User, Surrogate PK UUID)

Tên Cột	Kiểu Dữ Liệu	Khóa	Ràng Buộc
connection_id	UUID	PK	NOT NULL
user_id	-	FK, Unique	NOT NULL
github_user_id	-	-	NOT NULL
github_username	-	-	NOT NULL
access_token	-	-	NOT NULL
created_at	-	-	NOT NULL
updated_at	-	-	NOT NULL
Ràng buộc bổ sung: UNIQUE (user_id)			

6. Bảng user_notification_preferences (Bảng phụ đa trị từ NotificationPreferences của User)

Tên Cột	Kiểu Dữ Liệu	Khóa	Ràng Buộc
preference_id	UUID	PK	NOT NULL
user_id	-	FK	NOT NULL
channel	-	-	NOT NULL
Ràng buộc bổ sung: UNIQUE (user_id, channel) <i>Một người dùng chỉ có tối đa 1 bản ghi cho mỗi kênh.</i>			

7. Bảng github_connection_scopes (Bảng phụ đa trị từ scopes của GitHub-Connection)

Tên Cột	Kiểu Dữ Liệu	Khóa	Ràng Buộc
scope_id	UUID	PK	NOT NULL
connection_id	-	FK	NOT NULL
scope_value	-	-	NOT NULL
Ràng buộc bổ sung: UNIQUE (connection_id, scope_value) <i>Đảm bảo tài khoản OAuth không lưu lặp lại cùng một quyền scope.</i>			

8. Bảng workspace_members (Bảng trung gian mối kết hợp 3 ngôi User–Workspace–Role, tỷ lệ M:N:1)

Tên Cột	Kiểu Dữ Liệu	Khóa	Ràng Buộc
membership_id	UUID	PK	NOT NULL
user_id	-	FK	NOT NULL
workspace_id	-	FK	NOT NULL
role_id	-	FK	NOT NULL
joined_at	-	-	NOT NULL
status	-	-	NOT NULL
Ràng buộc bổ sung: UNIQUE (user_id, workspace_id) <i>Đảm bảo mỗi User chỉ giữ đúng 1 Role trong 1 Workspace (Scoped RBAC).</i>			

9. Bảng role_permissions (Bảng trung gian mối kết hợp Role–Permission, tỷ lệ M:N)

Tên Cột	Kiểu Dữ Liệu	Khóa	Ràng Buộc
role_permission_id	UUID	PK	NOT NULL
role_id	-	FK	NOT NULL
permission_id	-	FK	NOT NULL
granted_at	-	-	NOT NULL
Ràng buộc bổ sung: UNIQUE (role_id, permission_id) <i>Ngăn chặn gán trùng lặp quyền cho một vai trò (RBAC).</i>			

3.2.2 Phân hệ Mã nguồn & Cấu trúc Code

Tên Bảng: repositories

Tên Cột	Kiểu Dữ liệu	Khóa	Ràng Buộc
repository_id	UUID	PK	NOT NULL
workspace_id		FK	NOT NULL
name			NOT NULL
url		Unique	NOT NULL
default_branch			NOT NULL
sync_status			NOT NULL
last_synced_at			NULL
created_at			NOT NULL
updated_at			NOT NULL

Tên Bảng: branches

Tên Cột	Kiểu Dữ liệu	Khóa	Ràng Buộc
branch_id	UUID	PK	NOT NULL
repository_id		FK	NOT NULL
branch_name			NOT NULL
is_default			NOT NULL
created_at			NOT NULL
updated_at			NOT NULL

Tên Bảng: commits

Tên Cột	Kiểu Dữ liệu	Khóa	Ràng Buộc
commit_id	UUID	PK	NOT NULL
branch_id		FK	NOT NULL
user_id		FK	NULL
commit_hash		Unique	NOT NULL
message			NOT NULL
author_name			NOT NULL
author_email			NOT NULL
committed_at			NOT NULL
created_at			NOT NULL

Tên Bảng: pull_requests

Tên Cột	Kiểu Dữ liệu	Khóa	Ràng Buộc
pr_id	UUID	PK	NOT NULL
repository_id		FK	NOT NULL
pr_number			NOT NULL
title			NOT NULL
description			NOT NULL
source_branch			NOT NULL
target_branch			NOT NULL
status			NOT NULL
created_at			NOT NULL
updated_at			NOT NULL
merged_at			NULL

Tên Bảng: pull_request_labels

Tên Cột	Kiểu Dữ liệu	Khóa	Ràng Buộc
label_id	UUID	PK	NOT NULL
pr_id		FK	NOT NULL
label_name			NOT NULL

Tên Bảng: code_diffs

Tên Cột	Kiểu Dữ liệu	Khóa	Ràng Buộc
diff_id	UUID	PK	NOT NULL
commit_id		FK	NULL
pr_id		FK	NOT NULL
file_path			NOT NULL
change_type			NOT NULL
additions			NOT NULL
deletions			NOT NULL
diff_content			NULL
created_at			NOT NULL

Tên Bảng: files

Tên Cột	Kiểu Dữ liệu	Khóa	Ràng Buộc
file_id	UUID	PK	NOT NULL
repository_id		FK	NOT NULL
file_path			NOT NULL
file_type			NULL
status			NOT NULL
created_at			NOT NULL
updated_at			NOT NULL

Tên Bảng: code_entities

Tên Cột	Kiểu Dữ liệu	Khóa	Ràng Buộc
entity_id	UUID	PK	NOT NULL
file_id		FK	NOT NULL
entity_name			NOT NULL
entity_type			NOT NULL
signature			NULL
doc_comment			NULL
start_line			NOT NULL
end_line			NOT NULL
created_at			NOT NULL
updated_at			NOT NULL

Tên Bảng: dependencies

Tên Cột	Kiểu Dữ liệu	Khóa	Ràng Buộc
dependency_id	UUID	PK	NOT NULL
source_entity_id		FK	NOT NULL
target_entity_id		FK	NOT NULL
dependency_type			NOT NULL
created_at			NOT NULL

3.2.2 Phân hệ Mẫu & Tài liệu Cốt lõi

Bảng templates

Tên cột	Khóa	Ràng buộc
template_id	PK	not null
workspace_id	FK	not null
name		not null
description		null
content_structure		not null
creator_type		not null
status		not null
created_at		not null
updated_at		not null

Bảng template_sections

Tên cột	Khóa	Ràng buộc
section_id	PK	not null
template_id	FK	not null
heading_title		not null
order_index		not null
prompt_instruction		null
created_at		not null
updated_at		not null
Ràng buộc bổ sung: UNIQUE(template_id, order_index)		

Bảng placeholders

Tên cột	Khóa	Ràng buộc
placeholder_id	PK	not null
section_id	FK	not null
variable_name		not null
expected_data_type		not null
description		null
is_required		not null
created_at		not null
updated_at		not null
Ràng buộc bổ sung: UNIQUE(section_id, variable_name)		

Bảng template_mappings

Tên cột	Khóa	Ràng buộc
mapping_id	PK	not null
placeholder_id	FK	not null
entity_id	FK	not null
extraction_rule		null
is_active		not null
Ràng buộc bổ sung: UNIQUE(placeholder_id, entity_id)		

Bảng documents

Tên cột	Khóa	Ràng buộc
document_id	PK	not null
workspace_id	FK	not null
user_id	FK	not null
title		not null
document_type		not null
status		not null
created_at		not null
updated_at		not null
deleted_at		null

Bảng document_versions

Tên cột	Khóa	Ràng buộc
version_id	PK	not null
document_id	FK	not null
version_number		not null
content		not null
change_summary		null
status		not null
ticketed_codes		null
created_at		not null
updated_at		not null
Ràng buộc bổ sung: UNIQUE(document_id, version_number)		

Bảng change_logs

Tên cột	Khóa	Ràng buộc
log_id	PK	not null
version_id	FK	not null
change_type		not null
change_description		not null
diff_content		null
created_at		not null
updated_at		not null

3.2.2 Phân hệ minh chứng AI & Giám sát

Bảng review

Tên cột	Khóa	Ràng buộc
---------	------	-----------

review_id	PK	not null
version_id	FK	not null
user_id	FK	not null
status		not null
decision		null
summary_notes		null
created_at		not null
updated_at		not null

UNIQUE (version_id, user_id) Một người dùng chỉ được phép có tối đa 1 bản review cho mỗi phiên bản

Bảng review_comments

Tên cột	Khóa	Ràng buộc
comment_id	PK	not null
review_id	FK	not null
content		not null
is_resolved		not null, default false
created_at		not null
update_at		not null

Bảng grounding_evidences

Tên cột	Khóa	Ràng buộc
evidence_id	PK	not null
version_id	FK	not null
snippet_content		not null
purpose		null
created_at		not null

Bảng approvals

Tên cột	Khóa	Ràng buộc
approval_id	PK	not null
version_id	FK	not null
user_id	FK	not null
decision		not null
rejection_reason		null
approved_at		null

UNIQUE (version_id, user_id) Một người dùng chỉ nên có tối đa 1 bản ghi phê duyệt cho mỗi phiên bản.

Bảng drift_alerts

Tên cột	Khóa	Ràng buộc
alert_id	PK	not null
evidence_id	FK	not null
drift_type		not null
status		not null
detected_at		not null

UNIQUE (evidence_id, drift_type) Một loại drift chỉ nên có tối đa 1 cảnh báo cho mỗi bằng chứng.

Bảng grounding_evidence_code_entities

Tên cột	Khóa	Ràng buộc
link_id	PK	not null
evidence_id	FK	not null
entity_id	FK	not null
similarity_score		not null
match_type		not null

UNIQUE (evidence_id, entity_id) Một bằng chứng chỉ nên liên kết với một thực thể duy nhất.

Chương 4

Tổng kết Lược đồ Quan hệ Lô-gíc-Chuẩn hóa 3NF (Stage 2)

Sau khi áp dụng thành công **Thuật toán 2.2** (Ánh xạ mô hình ERD Quan niệm sang Lược đồ Quan hệ), toàn bộ thực thể, thuộc tính và mối kết hợp trong hệ thống đã được chuyển đổi hoàn chỉnh sang **Lược đồ ERD ở mức Lô-gíc (Logical ERD)**.

Đầu ra đạt được sau bước này bao gồm:

- **Cấu trúc bảng chuẩn hóa:** Chuyển đổi toàn bộ thực thể và các mối kết hợp $M : N$ thành **31 bảng** dữ liệu độc lập với tên gọi tuân thủ chuẩn `snake_case`.
- **Xử lý thuộc tính phức hợp & đa trị:** Giải quyết triệt để các thuộc tính phức hợp, thuộc tính đa trị và chuyển đổi các mối kết hợp 3 ngôi (như cụm `USER-WORKSPACE-ROLE`) thành các thực thể liên kết hợp lệ.
- **Cơ chế Khóa chính & Khóa ngoại:** Đã đồng bộ hóa việc sử dụng Khóa chính đơn *Surrogate PK (UUID)* cho tất cả các bảng, đồng thời thiết lập mạng lưới Khóa ngoại (*FK*) chặt chẽ để duy trì ràng buộc toàn vẹn tham chiếu.
- **Đạt dạng chuẩn 3NF/BCNF:** Xác định tập phụ thuộc hàm (FDs) cho 4 phân hệ nghiệp vụ, chứng minh toàn bộ các bảng loại bỏ được rủi ro dư thừa dữ liệu và bất thường thao tác.

Chi tiết sơ đồ ERD mức Lô-gíc và mô tả cấu trúc các bảng được trình bày ngay dưới đây:

Sau khi áp dụng thành công **Thuật toán 2.2** (Ánh xạ mô hình ERD Quan niệm sang Lược đồ Quan hệ), toàn bộ thực thể, thuộc tính và mối kết hợp trong

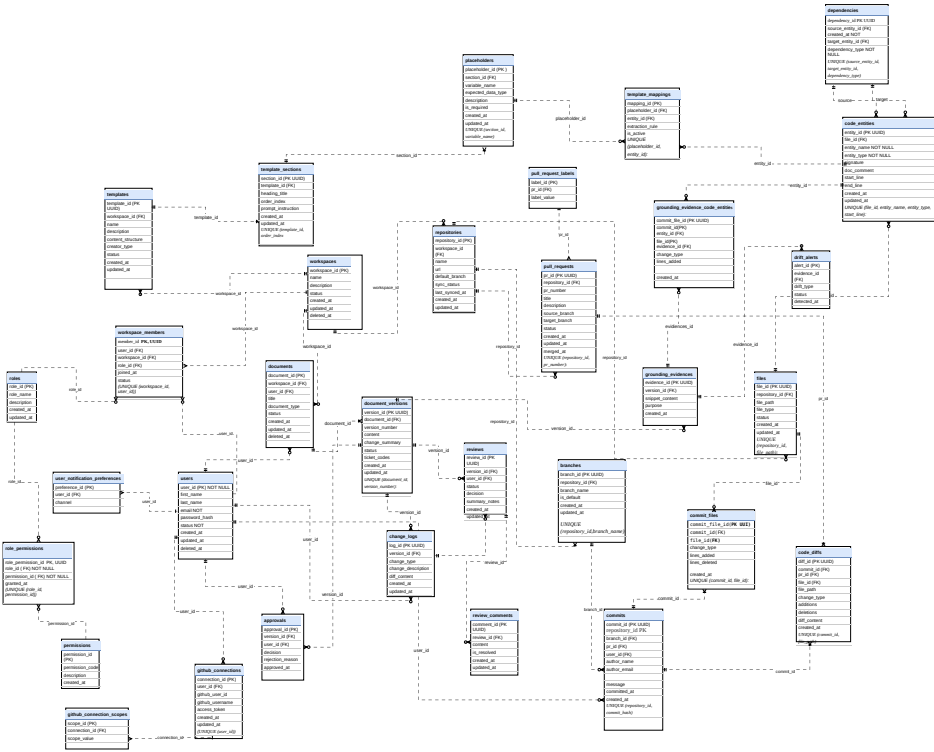
hệ thống đã được chuyển đổi hoàn chỉnh sang **Lược đồ ERD ở mức Lô-gíc (Logical ERD)**.

Đầu ra đạt được sau bước này bao gồm:

- **Cấu trúc bảng chuẩn hóa:** Chuyển đổi toàn bộ thực thể và các mối kết hợp $M : N$ thành **30 bảng** dữ liệu độc lập với tên gọi tuân thủ chuẩn `snake_case`.
- **Xử lý thuộc tính phức hợp & đa trị:** Giải quyết triệt để các thuộc tính phức hợp, thuộc tính đa trị và chuyển đổi các mối kết hợp 3 ngôi (như cụm `USER-WORKSPACE-ROLE`) thành các thực thể liên kết hợp lệ.
- **Cơ chế Khóa chính & Khóa ngoại:** Đã đồng bộ hóa việc sử dụng Khóa chính đơn *Surrogate PK (UUID)* cho tất cả các bảng, đồng thời thiết lập mạng lưới Khóa ngoại (*FK*) chặt chẽ để duy trì ràng buộc toàn vẹn tham chiếu.
- **Đạt dạng chuẩn 3NF/BCNF:** Xác định tập phụ thuộc hàm (FDs) cho 4 phân hệ nghiệp vụ, chứng minh toàn bộ các bảng loại bỏ được rủi ro dư thừa dữ liệu và bất thường thao tác.

Chi tiết sơ đồ ERD mức Lô-gíc và mô tả cấu trúc các bảng được trình bày ngay dưới đây:

4.1 Sơ đồ ERD Mức Logic Hệ thống LivingDocs



Hình 4.1: Sơ đồ ERD mức logic

Phần III

Thiết Kế Vật Lý

Chương 5

Thiết Kế CSDL Vật Lý & DDL SQL Theo Phân Hệ

5.1 Ánh Xạ Lược Đồ ER

5.1.1 Phân Hệ Quản Trị Xác Thực & Phân Quyền (Auth & RBAC Domain)

Mã DDL SQL Hoàn Chỉnh (schema_member_1.sql)

```
1
2 CREATE TABLE users (
3     user_id          UUID          NOT NULL DEFAULT
4     gen_random_uuid(),
5     email             VARCHAR(255)  NOT NULL,
6     password_hash     VARCHAR(255)  NULL,
7     first_name        VARCHAR(100)  NULL,
8     last_name         VARCHAR(100)  NULL,
9     status            VARCHAR(20)   NOT NULL,
10    created_at         TIMESTAMP     NOT NULL DEFAULT
11    CURRENT_TIMESTAMP,
12    updated_at         TIMESTAMP     NOT NULL DEFAULT
13    CURRENT_TIMESTAMP,
14    deleted_at         TIMESTAMP     NULL,
15    CONSTRAINT pk_users PRIMARY KEY (user_id),
16    CONSTRAINT uq_users_email UNIQUE (email)
17 );
```

```

17 CREATE TABLE workspaces (
18     workspace_id    UUID                NOT NULL DEFAULT
        gen_random_uuid(),
19     name            VARCHAR(150)        NOT NULL ,
20     description     TEXT                NULL ,
21     status          VARCHAR(20)         NOT NULL ,
22     created_at      TIMESTAMP           NOT NULL DEFAULT
        CURRENT_TIMESTAMP ,
23     updated_at      TIMESTAMP           NOT NULL DEFAULT
        CURRENT_TIMESTAMP ,
24     deleted_at      TIMESTAMP           NULL ,
25     CONSTRAINT pk_workspaces PRIMARY KEY (workspace_id)
26 );
27
28
29 CREATE TABLE roles (
30     role_id         UUID                NOT NULL DEFAULT
        gen_random_uuid(),
31     role_name       VARCHAR(50)         NOT NULL ,
32     description     TEXT                NULL ,
33     created_at      TIMESTAMP           NOT NULL DEFAULT
        CURRENT_TIMESTAMP ,
34     updated_at      TIMESTAMP           NOT NULL DEFAULT
        CURRENT_TIMESTAMP ,
35     CONSTRAINT pk_roles PRIMARY KEY (role_id)
36 );
37
38
39 CREATE TABLE permissions (
40     permission_id   UUID                NOT NULL DEFAULT
        gen_random_uuid(),
41     permission_code VARCHAR(50)         NOT NULL ,
42     description     TEXT                NULL ,
43     created_at      TIMESTAMP           NOT NULL DEFAULT
        CURRENT_TIMESTAMP ,
44     CONSTRAINT pk_permissions PRIMARY KEY (permission_id),
45     CONSTRAINT uq_permissions_code UNIQUE (permission_code)
46 );
47
48
49 CREATE TABLE github_connections (
50     connection_id   UUID                NOT NULL DEFAULT
        gen_random_uuid(),
51     user_id         UUID                NOT NULL ,
52     github_user_id  VARCHAR(50)         NOT NULL ,
53     github_username VARCHAR(100)        NOT NULL ,
54     access_token    VARCHAR(500)        NOT NULL ,
55     created_at      TIMESTAMP           NOT NULL DEFAULT
        CURRENT_TIMESTAMP ,

```

```

56     updated_at      TIMESTAMP      NOT NULL DEFAULT
CURRENT_TIMESTAMP ,
57     CONSTRAINT pk_github_connections PRIMARY KEY
(connection_id),
58     CONSTRAINT uq_github_connections_user UNIQUE (user_id),
59     CONSTRAINT fk_github_connections_user
60         FOREIGN KEY (user_id) REFERENCES users (user_id)
61         ON UPDATE CASCADE
62         ON DELETE CASCADE
63 );
64
65
66 CREATE TABLE user_notification_preferences (
67     preference_id    UUID           NOT NULL DEFAULT
gen_random_uuid(),
68     user_id          UUID           NOT NULL ,
69     channel          VARCHAR(20)    NOT NULL ,
70     CONSTRAINT pk_user_notification_preferences PRIMARY KEY
(preference_id),
71     CONSTRAINT fk_user_notification_pref_user
72         FOREIGN KEY (user_id) REFERENCES users (user_id)
73         ON UPDATE CASCADE
74         ON DELETE CASCADE
75 );
76
77
78 CREATE TABLE github_connection_scopes (
79     scope_id         UUID           NOT NULL DEFAULT
gen_random_uuid(),
80     connection_id    UUID           NOT NULL ,
81     scope_value      VARCHAR(50)    NOT NULL ,
82     CONSTRAINT pk_github_connection_scopes PRIMARY KEY
(scope_id),
83     CONSTRAINT fk_github_connection_scopes_conn
84         FOREIGN KEY (connection_id) REFERENCES
github_connections (connection_id)
85         ON UPDATE CASCADE
86         ON DELETE CASCADE
87 );
88
89 CREATE TABLE workspace_members (
90     membership_id    UUID           NOT NULL DEFAULT
gen_random_uuid(),
91     user_id          UUID           NOT NULL ,
92     workspace_id     UUID           NOT NULL ,
93     role_id          UUID           NOT NULL ,
94     joined_at        TIMESTAMP      NOT NULL DEFAULT
CURRENT_TIMESTAMP ,
95     status           VARCHAR(20)    NOT NULL ,

```

```

96     CONSTRAINT pk_workspace_members PRIMARY KEY
    (membership_id),
97
98     CONSTRAINT uq_workspace_members_user_workspace UNIQUE
    (user_id, workspace_id),
99     CONSTRAINT fk_workspace_members_user
100         FOREIGN KEY (user_id) REFERENCES users (user_id)
101         ON UPDATE CASCADE
102         ON DELETE CASCADE,
103     CONSTRAINT fk_workspace_members_workspace
104         FOREIGN KEY (workspace_id) REFERENCES workspaces
    (workspace_id)
105         ON UPDATE CASCADE
106         ON DELETE CASCADE,
107     CONSTRAINT fk_workspace_members_role
108         FOREIGN KEY (role_id) REFERENCES roles (role_id)
109         ON UPDATE CASCADE
110         ON DELETE RESTRICT
111 );
112
113
114 CREATE TABLE role_permissions (
115     role_permission_id UUID          NOT NULL DEFAULT
    gen_random_uuid(),
116     role_id             UUID          NOT NULL,
117     permission_id       UUID          NOT NULL,
118     granted_at          TIMESTAMP     NOT NULL DEFAULT
    CURRENT_TIMESTAMP,
119     CONSTRAINT pk_role_permissions PRIMARY KEY
    (role_permission_id),
120     CONSTRAINT uq_role_permissions_pair UNIQUE (role_id,
    permission_id),
121     CONSTRAINT fk_role_permissions_role
122         FOREIGN KEY (role_id) REFERENCES roles (role_id)
123         ON UPDATE CASCADE
124         ON DELETE CASCADE,
125     CONSTRAINT fk_role_permissions_permission
126         FOREIGN KEY (permission_id) REFERENCES permissions
    (permission_id)
127         ON UPDATE CASCADE
128         ON DELETE CASCADE
129 );
130
131
132 CREATE INDEX idx_github_connections_user_id ON
    github_connections(user_id);
133 CREATE INDEX idx_workspace_members_workspace_id ON
    workspace_members(workspace_id);

```

```

134 CREATE INDEX idx_workspace_members_user_id ON
    workspace_members(user_id);
135 CREATE INDEX idx_role_permissions_role_id ON
    role_permissions(role_id);
136
137
138 CREATE OR REPLACE FUNCTION set_updated_at()
139 RETURNS TRIGGER AS $$
140 BEGIN
141     NEW.updated_at := CURRENT_TIMESTAMP;
142     RETURN NEW;
143 END;
144
145
146 CREATE TRIGGER trg_users_updated_at
147 BEFORE UPDATE ON users
148 FOR EACH ROW EXECUTE FUNCTION set_updated_at();
149
150 CREATE TRIGGER trg_workspaces_updated_at
151 BEFORE UPDATE ON workspaces
152 FOR EACH ROW EXECUTE FUNCTION set_updated_at();
153
154 CREATE TRIGGER trg_roles_updated_at
155 BEFORE UPDATE ON roles
156 FOR EACH ROW EXECUTE FUNCTION set_updated_at();
157
158 CREATE TRIGGER trg_github_connections_updated_at
159 BEFORE UPDATE ON github_connections
160 FOR EACH ROW EXECUTE FUNCTION set_updated_at();
161
162 GO

```

5.1.2 Ánh Xạ Lược Đồ ER Phân Hệ Không Gian Làm Việc & Tích Hợp VCS (Workspace & External VCS Domain)

Mã DDL SQL Hoàn Chỉnh (schema_member_workspace.sql)

```

1 -- 1. BANG WORKSPACES (Quan ly khong gian lam viec cua nguoi
    dung)
2 CREATE TABLE workspaces (
3     workspace_id UNIQUEIDENTIFIER PRIMARY KEY DEFAULT NEWID(),
4     name VARCHAR(150) NOT NULL,
5     description TEXT NULL,
6     status VARCHAR(20) NOT NULL DEFAULT 'ACTIVE',
7     created_at DATETIMEOFFSET DEFAULT SYSDATETIMEOFFSET(),
8     updated_at DATETIMEOFFSET DEFAULT SYSDATETIMEOFFSET(),
9     deleted_at DATETIMEOFFSET NULL,

```

```

10
11 -- Constraints
12 CONSTRAINT uk_workspace_id_name UNIQUE (workspace_id, name)
13 ,
14 CONSTRAINT chk_workspace_status CHECK (status IN ('ACTIVE',
15 'ARCHIVED'))
16 );
17
18 -- 2. BANG GITHUB_CONNECTIONS (Ket noi tai khoan GitHub voi
19 Workspace)
20 CREATE TABLE github_connections (
21 connection_id UNIQUEIDENTIFIER PRIMARY KEY DEFAULT NEWID(),
22 workspace_id UNIQUEIDENTIFIER NOT NULL,
23 provider VARCHAR(50) NOT NULL DEFAULT 'GITHUB',
24 account_name VARCHAR(100) NOT NULL,
25 access_token TEXT NOT NULL,
26 sync_status VARCHAR(20) NOT NULL DEFAULT 'IDLE',
27 created_at DATETIME2 NOT NULL DEFAULT GETDATE(),
28 updated_at DATETIME2 NULL,
29
30 -- Constraints
31 CONSTRAINT uk_github_connection UNIQUE (connection_id,
32 provider),
33 CONSTRAINT chk_sync_status CHECK (sync_status IN ('IDLE', '
34 SYNCING', 'SUCCESS', 'FAILED'))
35 );
36
37 -- 3. BANG GITHUB_CONNECTION_SCOPES (Pham vi quyen han cua ket
38 noi GitHub)
39 CREATE TABLE github_connection_scopes (
40 scope_id UNIQUEIDENTIFIER PRIMARY KEY DEFAULT NEWID(),
41 connection_id UNIQUEIDENTIFIER NOT NULL,
42 scope_value VARCHAR(100) NOT NULL,
43 created_at DATETIME2 NOT NULL DEFAULT GETDATE(),
44
45 CONSTRAINT uk_connection_scope UNIQUE (connection_id,
46 scope_value)
47 );
48
49 -- 4. BANG REPOSITORIES (Kho luu tru ma nguon lien ket)
50 CREATE TABLE repositories (
51 repository_id UNIQUEIDENTIFIER PRIMARY KEY DEFAULT NEWID(),
52 workspace_id UNIQUEIDENTIFIER NOT NULL,
53 name VARCHAR(150) NOT NULL,
54 url VARCHAR(255) NOT NULL,
55 default_branch VARCHAR(50) DEFAULT 'main',
56 created_at DATETIME2 NOT NULL DEFAULT GETDATE(),
57 updated_at DATETIME2 NULL,
58

```

```

52     CONSTRAINT uk_repository_name UNIQUE (repository_id, name)
53 );
54
55 -- 5. BANG BRANCHES (Cac nhanh lam viec trong Repository)
56 CREATE TABLE branches (
57     branch_id UNIQUEIDENTIFIER PRIMARY KEY DEFAULT NEWID(),
58     repository_id UNIQUEIDENTIFIER NOT NULL,
59     name VARCHAR(100) NOT NULL,
60     is_protected BIT NOT NULL DEFAULT 0,
61     created_at DATETIME2 NOT NULL DEFAULT GETDATE(),
62
63     CONSTRAINT uk_branch_name UNIQUE (repository_id, name)
64 );
65 GO
66
67 -- =====
68 -- KHAI BAO CAC KHOA NGOAI THAM CHIEU (CROSS-DOMAIN & INTERNAL
69 -- FKs)
70 -- =====
71 ALTER TABLE github_connections ADD CONSTRAINT
72     fk_github_connections_workspace
73     FOREIGN KEY (workspace_id) REFERENCES workspaces(
74         workspace_id) ON DELETE CASCADE;
75
76 ALTER TABLE github_connection_scopes ADD CONSTRAINT
77     fk_github_scopes_connection
78     FOREIGN KEY (connection_id) REFERENCES github_connections(
79         connection_id) ON DELETE CASCADE;
80
81 ALTER TABLE repositories ADD CONSTRAINT
82     fk_repositories_workspace
83     FOREIGN KEY (workspace_id) REFERENCES workspaces(
84         workspace_id) ON DELETE CASCADE;
85
86 ALTER TABLE branches ADD CONSTRAINT fk_branches_repository
87     FOREIGN KEY (repository_id) REFERENCES repositories(
88         repository_id) ON DELETE CASCADE;
89 GO

```

5.1.3 Cấu Trúc Mã Nguồn & Phân Tích AST (Code Structure & AST Analysis Domain)

Mã DDL SQL Hoàn Chỉnh (schema_member_3.sql)

```

1
2 CREATE TABLE commits (
3     commit_id UNIQUEIDENTIFIER PRIMARY KEY DEFAULT NEWID(),
4     repository_id UNIQUEIDENTIFIER NOT NULL,

```

```

5      branch_id UNIQUEIDENTIFIER NOT NULL,
6      pr_id UNIQUEIDENTIFIER NULL,
7      commit_hash VARCHAR(255) NOT NULL,
8      author_name VARCHAR(255) NULL,
9      author_email VARCHAR(255) NULL,
10     message NVARCHAR(MAX) NULL,
11     committed_at DATETIME2 NOT NULL,
12     created_at DATETIME2 NOT NULL DEFAULT GETDATE(),
13     updated_at DATETIME2 NULL,
14
15     CONSTRAINT uq_commits_repository_hash
16         UNIQUE (repository_id, commit_hash)
17 );
18
19 CREATE INDEX idx_commits_repository_id
20     ON commits(repository_id);
21
22 CREATE INDEX idx_commits_branch_id
23     ON commits(branch_id);
24
25 CREATE INDEX idx_commits_pr_id
26     ON commits(pr_id);
27
28 CREATE INDEX idx_commits_committed_at
29     ON commits(committed_at);
30
31 CREATE TABLE pull_requests (
32     pr_id UNIQUEIDENTIFIER PRIMARY KEY DEFAULT NEWID(),
33     repository_id UNIQUEIDENTIFIER NOT NULL,
34     pr_number INT NOT NULL,
35     title VARCHAR(500) NOT NULL,
36     description NVARCHAR(MAX) NULL,
37     source_branch VARCHAR(255) NULL,
38     target_branch VARCHAR(255) NULL,
39     status VARCHAR(20) NOT NULL DEFAULT 'OPEN',
40     created_at DATETIME2 NOT NULL DEFAULT GETDATE(),
41     updated_at DATETIME2 NULL,
42     merged_at DATETIME2 NULL,
43
44     CONSTRAINT uq_pull_requests_repository_pr
45         UNIQUE (repository_id, pr_number),
46
47     CONSTRAINT chk_pull_requests_status
48         CHECK (status IN ('OPEN', 'CLOSED', 'MERGED'))
49 );
50
51 CREATE INDEX idx_pull_requests_repository_id
52     ON pull_requests(repository_id);
53

```

```

54 CREATE INDEX idx_pull_requests_status
55     ON pull_requests(status);
56
57 CREATE INDEX idx_pull_requests_created_at
58     ON pull_requests(created_at);
59
60 CREATE TABLE pull_request_labels (
61     pull_request_label_id UNIQUEIDENTIFIER PRIMARY KEY DEFAULT
        NEWID(),
62     pr_id UNIQUEIDENTIFIER NOT NULL,
63     label_name VARCHAR(100) NOT NULL,
64     label_color VARCHAR(20) NULL,
65
66     CONSTRAINT uq_pull_request_labels
67         UNIQUE (pr_id, label_name)
68 );
69
70 CREATE INDEX idx_pull_request_labels_pr_id
71     ON pull_request_labels(pr_id);
72
73 CREATE INDEX idx_pull_request_labels_name
74     ON pull_request_labels(label_name);
75
76 CREATE TABLE files (
77     file_id UNIQUEIDENTIFIER PRIMARY KEY DEFAULT NEWID(),
78     repository_id UNIQUEIDENTIFIER NOT NULL,
79     file_path VARCHAR(1000) NOT NULL,
80     file_type VARCHAR(50) NULL,
81     status VARCHAR(20) NOT NULL DEFAULT 'ACTIVE',
82     created_at DATETIME2 NOT NULL DEFAULT GETDATE(),
83     updated_at DATETIME2 NULL,
84
85     CONSTRAINT uq_files_repository_file_path
86         UNIQUE (repository_id, file_path),
87
88     CONSTRAINT chk_files_status
89         CHECK (status IN ('ACTIVE', 'DELETED', 'RENAMED'))
90 );
91
92 CREATE INDEX idx_files_repository_id
93     ON files(repository_id);
94
95 CREATE INDEX idx_files_file_path
96     ON files(file_path);
97
98 CREATE INDEX idx_files_file_type
99     ON files(file_type);
100
101 CREATE INDEX idx_files_status

```

```

102     ON files(status);
103
104 CREATE TABLE code_diffs (
105     diff_id UNIQUEIDENTIFIER PRIMARY KEY DEFAULT NEWID(),
106     commit_id UNIQUEIDENTIFIER NOT NULL,
107     pr_id UNIQUEIDENTIFIER NULL,
108     file_id UNIQUEIDENTIFIER NOT NULL,
109     file_path VARCHAR(1000) NULL,
110     change_type VARCHAR(20) NOT NULL,
111     additions INT NULL,
112     deletions INT NULL,
113     diff_content NVARCHAR(MAX) NULL,
114     created_at DATETIME2 NOT NULL DEFAULT GETDATE(),
115
116     CONSTRAINT chk_code_diffs_change_type
117         CHECK (change_type IN (
118             'ADDED',
119             'MODIFIED',
120             'DELETED',
121             'RENAMED'
122         )),
123
124     CONSTRAINT uq_code_diffs_commit_file
125         UNIQUE (commit_id, file_id)
126 );
127
128 CREATE INDEX idx_code_diffs_commit_id
129     ON code_diffs(commit_id);
130
131 CREATE INDEX idx_code_diffs_pr_id
132     ON code_diffs(pr_id);
133
134 CREATE INDEX idx_code_diffs_file_id
135     ON code_diffs(file_id);
136
137 CREATE INDEX idx_code_diffs_change_type
138     ON code_diffs(change_type);
139
140 CREATE TABLE code_entities (
141     code_entity_id UNIQUEIDENTIFIER PRIMARY KEY DEFAULT
142     NEWID(),
143     file_id UNIQUEIDENTIFIER NOT NULL,
144     entity_name VARCHAR(255) NOT NULL,
145     entity_type VARCHAR(50) NOT NULL,
146     signature NVARCHAR(MAX) NULL,
147     doc_comment NVARCHAR(MAX) NULL,
148     start_line INT NULL,
149     end_line INT NULL,
150     created_at DATETIME2 NOT NULL DEFAULT GETDATE(),

```

```

150     updated_at DATETIME2 NULL,
151
152     CONSTRAINT chk_code_entities_entity_type
153         CHECK (entity_type IN (
154             'CLASS',
155             'INTERFACE',
156             'FUNCTION',
157             'METHOD',
158             'VARIABLE',
159             'CONSTANT',
160             'MODULE',
161             'PACKAGE',
162             'ENUM',
163             'STRUCT'
164         )),
165
166     CONSTRAINT chk_code_entities_line_range
167         CHECK (
168             start_line IS NULL
169             OR end_line IS NULL
170             OR start_line <= end_line
171         ),
172
173     CONSTRAINT uq_code_entities
174         UNIQUE (file_id, entity_name, entity_type, start_line)
175 );
176
177 CREATE INDEX idx_code_entities_file_id
178     ON code_entities(file_id);
179
180 CREATE INDEX idx_code_entities_entity_name
181     ON code_entities(entity_name);
182
183 CREATE INDEX idx_code_entities_entity_type
184     ON code_entities(entity_type);
185
186 CREATE TABLE dependencies (
187     dependency_id UNIQUEIDENTIFIER PRIMARY KEY DEFAULT NEWID(),
188     source_entity_id UNIQUEIDENTIFIER NOT NULL,
189     target_entity_id UNIQUEIDENTIFIER NOT NULL,
190     dependency_type VARCHAR(50) NOT NULL,
191     created_at DATETIME2 NOT NULL DEFAULT GETDATE(),
192
193     CONSTRAINT uq_dependencies
194         UNIQUE (
195             source_entity_id,
196             target_entity_id,
197             dependency_type
198         ),

```

```

199
200     CONSTRAINT chk_dependencies_dependency_type
201         CHECK (dependency_type IN (
202             'CALLS',
203             'IMPORTS',
204             'INHERITS',
205             'IMPLEMENTS',
206             'USES',
207             'REFERENCES',
208             'DEPENDS_ON'
209         )),
210
211     CONSTRAINT chk_dependencies_no_self_reference
212         CHECK (source_entity_id <> target_entity_id)
213 );
214
215 CREATE INDEX idx_dependencies_source_entity_id
216     ON dependencies(source_entity_id);
217
218 CREATE INDEX idx_dependencies_target_entity_id
219     ON dependencies(target_entity_id);
220
221 CREATE INDEX idx_dependencies_dependency_type
222     ON dependencies(dependency_type);
223
224 GO
225
226 ALTER TABLE commits ADD CONSTRAINT fk_commits_repository
227     FOREIGN KEY (repository_id)
228     REFERENCES repositories(repository_id);
229
230 ALTER TABLE commits ADD CONSTRAINT fk_commits_branch
231     FOREIGN KEY (branch_id)
232     REFERENCES branches(branch_id);
233
234 ALTER TABLE commits ADD CONSTRAINT fk_commits_pull_request
235     FOREIGN KEY (pr_id)
236     REFERENCES pull_requests(pr_id)
237     ON DELETE SET NULL;
238
239
240 -- PULL REQUESTS
241
242 ALTER TABLE pull_requests ADD CONSTRAINT
243     fk_pull_requests_repository
244     FOREIGN KEY (repository_id)
245     REFERENCES repositories(repository_id);
246

```

```

247 -- PULL REQUEST LABELS
248
249 ALTER TABLE pull_request_labels ADD CONSTRAINT
      fk_pull_request_labels_pr
250     FOREIGN KEY (pr_id)
251     REFERENCES pull_requests(pr_id)
252     ON DELETE CASCADE;
253
254
255 -- FILES
256
257 ALTER TABLE files ADD CONSTRAINT fk_files_repository
258     FOREIGN KEY (repository_id)
259     REFERENCES repositories(repository_id)
260     ON DELETE CASCADE;
261
262
263 -- CODE DIFFS
264
265 ALTER TABLE code_diffs ADD CONSTRAINT fk_code_diffs_commit
266     FOREIGN KEY (commit_id)
267     REFERENCES commits(commit_id)
268     ON DELETE CASCADE;
269
270 ALTER TABLE code_diffs ADD CONSTRAINT
      fk_code_diffs_pull_request
271     FOREIGN KEY (pr_id)
272     REFERENCES pull_requests(pr_id)
273     ON DELETE SET NULL;
274
275 ALTER TABLE code_diffs ADD CONSTRAINT fk_code_diffs_file
276     FOREIGN KEY (file_id)
277     REFERENCES files(file_id)
278     ON DELETE CASCADE;
279
280
281 -- CODE ENTITIES
282
283 ALTER TABLE code_entities ADD CONSTRAINT fk_code_entities_file
284     FOREIGN KEY (file_id)
285     REFERENCES files(file_id)
286     ON DELETE CASCADE;
287
288
289 -- DEPENDENCIES
290
291 ALTER TABLE dependencies ADD CONSTRAINT
      fk_dependencies_source_entity
292     FOREIGN KEY (source_entity_id)

```

```

293 REFERENCES code_entities (code_entity_id)
294 ON DELETE CASCADE;
295
296 ALTER TABLE dependencies ADD CONSTRAINT
297     fk_dependencies_target_entity
298     FOREIGN KEY (target_entity_id)
299     REFERENCES code_entities (code_entity_id)
300     ON DELETE CASCADE;
301 GO

```

5.1.4 Phân Hệ Khuôn Mẫu & Quản Lý Vòng Đời Tài Liệu (Template & Core Documentation Domain)

Mã DDL SQL Hoàn Chỉnh (schema_member_4.sql)

```

1
2
3
4 CREATE TABLE templates (
5     template_id          UUID          NOT NULL DEFAULT
6     gen_random_uuid(),
7     workspace_id         UUID          NOT NULL,
8     name                 VARCHAR(150)  NOT NULL,
9     description          TEXT          NULL,
10    content_structure     TEXT          NULL,
11    creator_type          VARCHAR(20)   NOT NULL,
12    status                VARCHAR(20)   NOT NULL DEFAULT
13    'DRAFT',
14    created_at            TIMESTAMP     NOT NULL DEFAULT
15    CURRENT_TIMESTAMP,
16    updated_at            TIMESTAMP     NOT NULL DEFAULT
17    CURRENT_TIMESTAMP,
18    CONSTRAINT pk_templates PRIMARY KEY (template_id),
19
20    CONSTRAINT uq_templates_workspace_name UNIQUE
21    (workspace_id, name),
22    CONSTRAINT fk_templates_workspace
23    FOREIGN KEY (workspace_id) REFERENCES workspaces
24    (workspace_id)
25    ON UPDATE CASCADE
26    ON DELETE CASCADE,
27
28    CONSTRAINT ck_templates_creator_type
29    CHECK (creator_type IN ('SYSTEM', 'USER', 'AI')),
30    CONSTRAINT ck_templates_status
31    CHECK (status IN ('DRAFT', 'ACTIVE', 'ARCHIVED')),
32    CONSTRAINT ck_templates_name_not_blank
33    CHECK (length(btrim(name)) > 0),

```

```

27     CONSTRAINT ck_templates_updated_after_created
28         CHECK (updated_at >= created_at)
29 );
30
31 CREATE TABLE template_sections (
32     section_id          UUID          NOT NULL DEFAULT
33     gen_random_uuid(),
34     template_id         UUID          NOT NULL,
35     heading_title       VARCHAR(255)  NOT NULL,
36     order_index         INTEGER       NOT NULL,
37     prompt_instruction  TEXT          NULL,
38     created_at          TIMESTAMP     NOT NULL DEFAULT
39     CURRENT_TIMESTAMP,
40     updated_at          TIMESTAMP     NOT NULL DEFAULT
41     CURRENT_TIMESTAMP,
42     CONSTRAINT pk_template_sections PRIMARY KEY (section_id),
43     CONSTRAINT uq_template_sections_order UNIQUE (template_id,
44     order_index),
45     CONSTRAINT fk_template_sections_template
46     FOREIGN KEY (template_id) REFERENCES templates
47     (template_id)
48     ON UPDATE CASCADE
49     ON DELETE CASCADE,
50     CONSTRAINT ck_template_sections_order_index
51     CHECK (order_index >= 0),
52     CONSTRAINT ck_template_sections_heading_not_blank
53     CHECK (length(btrim(heading_title)) > 0)
54 );
55
56 CREATE TABLE placeholders (
57     placeholder_id      UUID          NOT NULL DEFAULT
58     gen_random_uuid(),
59     section_id          UUID          NOT NULL,
60     variable_name       VARCHAR(100)  NOT NULL,
61     expected_data_type  VARCHAR(30)   NOT NULL,
62     description         TEXT          NULL,
63     is_required         BOOLEAN       NOT NULL DEFAULT TRUE,
64     created_at          TIMESTAMP     NOT NULL DEFAULT
65     CURRENT_TIMESTAMP,
66     updated_at          TIMESTAMP     NOT NULL DEFAULT
67     CURRENT_TIMESTAMP,
68     CONSTRAINT pk_placeholders PRIMARY KEY (placeholder_id),
69     CONSTRAINT uq_placeholders_section_variable UNIQUE
70     (section_id, variable_name),
71     CONSTRAINT fk_placeholders_section
72     FOREIGN KEY (section_id) REFERENCES template_sections
73     (section_id)

```

```

66         ON UPDATE CASCADE
67         ON DELETE CASCADE,
68     CONSTRAINT ck_placeholders_expected_data_type
69         CHECK (expected_data_type IN ('STRING', 'NUMBER',
70         'BOOLEAN',
71                                     'DATE', 'LIST',
72         'OBJECT', 'CODE')),
73     CONSTRAINT ck_placeholders_variable_name_format
74         CHECK (variable_name ~ '^[A-Za-z_][A-Za-z0-9_]*$')
75 );
76 CREATE TABLE template_mappings (
77     mapping_id          UUID          NOT NULL DEFAULT
78     gen_random_uuid(),
79     placeholder_id      UUID          NOT NULL,
80     code_entity_id      UUID          NOT NULL,
81     extraction_rule     TEXT          NULL,
82     is_active           BOOLEAN       NOT NULL DEFAULT TRUE,
83     CONSTRAINT pk_template_mappings PRIMARY KEY (mapping_id),
84     CONSTRAINT uq_template_mappings_pair UNIQUE
85     (placeholder_id, code_entity_id),
86     CONSTRAINT fk_template_mappings_placeholder
87     FOREIGN KEY (placeholder_id) REFERENCES placeholders
88     (placeholder_id)
89     ON UPDATE CASCADE
90     ON DELETE CASCADE,
91     CONSTRAINT fk_template_mappings_code_entity
92     FOREIGN KEY (code_entity_id) REFERENCES code_entities
93     (code_entity_id)
94     ON UPDATE CASCADE
95     ON DELETE CASCADE
96 );
97 CREATE TABLE documents (
98     document_id         UUID          NOT NULL DEFAULT
99     gen_random_uuid(),
100    workspace_id        UUID          NOT NULL,
101    author_user_id      UUID          NOT NULL,
102    title               VARCHAR(255)   NOT NULL,
103    document_type       VARCHAR(30)    NOT NULL,
104    status              VARCHAR(20)    NOT NULL DEFAULT
105    'DRAFT',
106    created_at          TIMESTAMP      NOT NULL DEFAULT
107    CURRENT_TIMESTAMP,
108    updated_at          TIMESTAMP      NOT NULL DEFAULT
109    CURRENT_TIMESTAMP,
110    deleted_at          TIMESTAMP      NULL,

```

```

105     CONSTRAINT pk_documents PRIMARY KEY (document_id),
106     CONSTRAINT fk_documents_workspace
107         FOREIGN KEY (workspace_id) REFERENCES workspaces
108         (workspace_id)
109         ON UPDATE CASCADE
110         ON DELETE CASCADE,
111     CONSTRAINT fk_documents_author
112         FOREIGN KEY (author_user_id) REFERENCES users (user_id)
113         ON UPDATE CASCADE
114         ON DELETE RESTRICT,
115     CONSTRAINT ck_documents_type
116         CHECK (document_type IN ('SRS', 'API_DOC',
117         'USER_GUIDE',
118         'RELEASE_NOTE',
119         'TECH_DESIGN', 'OTHER')),
120     CONSTRAINT ck_documents_status
121         CHECK (status IN ('DRAFT', 'IN_REVIEW', 'PUBLISHED',
122         'ARCHIVED')),
123     CONSTRAINT ck_documents_title_not_blank
124         CHECK (length(btrim(title)) > 0),
125     CONSTRAINT ck_documents_deleted_after_created
126         CHECK (deleted_at IS NULL OR deleted_at >= created_at)
127 );
128
129 CREATE UNIQUE INDEX uq_documents_workspace_title_active
130 ON documents (workspace_id, title)
131 WHERE deleted_at IS NULL;
132
133 CREATE TABLE document_versions (
134     version_id          UUID          NOT NULL DEFAULT
135     gen_random_uuid(),
136     document_id         UUID          NOT NULL,
137     version_number      VARCHAR(20)   NOT NULL,
138     content             TEXT          NULL,
139     change_summary      TEXT          NULL,
140     status              VARCHAR(20)   NOT NULL DEFAULT
141     'DRAFT',
142     ticket_codes        VARCHAR(500)  NULL,
143     created_at          TIMESTAMP     NOT NULL DEFAULT
144     CURRENT_TIMESTAMP,
145     updated_at          TIMESTAMP     NOT NULL DEFAULT
146     CURRENT_TIMESTAMP,
147     CONSTRAINT pk_document_versions PRIMARY KEY (version_id),
148
149     CONSTRAINT uq_document_versions_number UNIQUE
150     (document_id, version_number),
151     CONSTRAINT fk_document_versions_document
152         FOREIGN KEY (document_id) REFERENCES documents
153         (document_id)

```

```

144         ON UPDATE CASCADE
145         ON DELETE CASCADE,
146     CONSTRAINT ck_document_versions_status
147     CHECK (status IN ('DRAFT', 'IN_REVIEW', 'APPROVED',
148     'REJECTED', 'PUBLISHED')),
149
149     CONSTRAINT ck_document_versions_number_format
150     CHECK (version_number ~ '^[vV]?[0-9]+(\.[0-9]+){0,2}$')
151 );
152
153 CREATE TABLE change_logs (
154     log_id          UUID          NOT NULL DEFAULT
155     gen_random_uuid(),
156     version_id      UUID          NOT NULL,
157     change_type     VARCHAR(20)   NOT NULL,
158     change_description TEXT       NULL,
159     diff_content    TEXT          NULL,
160     created_at      TIMESTAMP     NOT NULL DEFAULT
161     CURRENT_TIMESTAMP,
162     updated_at      TIMESTAMP     NOT NULL DEFAULT
163     CURRENT_TIMESTAMP,
164     CONSTRAINT pk_change_logs PRIMARY KEY (log_id),
165     CONSTRAINT fk_change_logs_version
166     FOREIGN KEY (version_id) REFERENCES document_versions
167     (version_id)
168     ON UPDATE CASCADE
169     ON DELETE CASCADE,
170     CONSTRAINT ck_change_logs_change_type
171     CHECK (change_type IN ('CREATE', 'UPDATE', 'DELETE',
172     'RESTORE', 'PUBLISH'))
173 );
174
175 CREATE INDEX idx_templates_workspace_id ON templates
176     (workspace_id);
177
178 CREATE INDEX idx_templates_status ON templates (status);
179
180 CREATE INDEX idx_template_mappings_code_entity ON
181     template_mappings (code_entity_id);
182
183 CREATE INDEX idx_template_mappings_active ON
184     template_mappings (placeholder_id)
185     WHERE is_active = TRUE;
186
187 CREATE INDEX idx_documents_workspace_id ON documents
188     (workspace_id);
189
190 CREATE INDEX idx_documents_author ON documents
191     (author_user_id);
192
193 CREATE INDEX idx_documents_status ON documents (status);
194
195 CREATE INDEX idx_documents_created_at ON documents

```

```

182         (created_at DESC);
183 CREATE INDEX idx_document_versions_status ON
184     document_versions (status);
185 CREATE INDEX idx_document_versions_created_at ON
186     document_versions (document_id, created_at DESC);
187 CREATE INDEX idx_change_logs_version_id ON change_logs
188     (version_id);
189 CREATE INDEX idx_change_logs_created_at ON change_logs
190     (created_at DESC);
191
192 CREATE OR REPLACE FUNCTION set_updated_at()
193 RETURNS TRIGGER AS $$
194 BEGIN
195     NEW.updated_at := CURRENT_TIMESTAMP;
196     RETURN NEW;
197 END;
198
199 CREATE TRIGGER trg_templates_updated_at
200 BEFORE UPDATE ON templates
201 FOR EACH ROW EXECUTE FUNCTION set_updated_at();
202 CREATE TRIGGER trg_template_sections_updated_at
203 BEFORE UPDATE ON template_sections
204 FOR EACH ROW EXECUTE FUNCTION set_updated_at();
205 CREATE TRIGGER trg_placeholders_updated_at
206 BEFORE UPDATE ON placeholders
207 FOR EACH ROW EXECUTE FUNCTION set_updated_at();
208 CREATE TRIGGER trg_documents_updated_at
209 BEFORE UPDATE ON documents
210 FOR EACH ROW EXECUTE FUNCTION set_updated_at();
211 CREATE TRIGGER trg_document_versions_updated_at
212 BEFORE UPDATE ON document_versions
213 FOR EACH ROW EXECUTE FUNCTION set_updated_at();
214 CREATE TRIGGER trg_change_logs_updated_at
215 BEFORE UPDATE ON change_logs
216 FOR EACH ROW EXECUTE FUNCTION set_updated_at();
217 GO

```

5.1.5 nh Xạ Lược Đồ ER Phân Hệ Kiểm Duyệt, Bằng Chứng AI & Giám Sát Hệ Thống (Governance, AI Evidence & Observability Domain)

Mã DDL SQL Hoàn Chỉnh (schema_member_5.sql)

```

2
3 CREATE TABLE reviews (
4     review_id UNIQUEIDENTIFIER PRIMARY KEY DEFAULT NEWID(),
5     version_id UNIQUEIDENTIFIER NOT NULL,
6     user_id UNIQUEIDENTIFIER NOT NULL,
7     status VARCHAR(20) NOT NULL DEFAULT 'PENDING',
8     decision VARCHAR(20) NULL,
9     summary_notes NVARCHAR(MAX) NULL,
10    created_at DATETIME2 NOT NULL DEFAULT GETDATE(),
11    updated_at DATETIME2 NULL,
12
13    CONSTRAINT chk_reviews_status CHECK (status IN ('PENDING',
14    'IN_PROGRESS', 'COMPLETED')),
15    CONSTRAINT chk_reviews_decision CHECK (decision IS NULL OR
16    decision IN ('APPROVED', 'REJECTED', 'CHANGES_REQUESTED'))
17 );
18
19 CREATE TABLE review_comments (
20     comment_id UNIQUEIDENTIFIER PRIMARY KEY DEFAULT NEWID(),
21     review_id UNIQUEIDENTIFIER NOT NULL,
22     user_id UNIQUEIDENTIFIER NOT NULL,
23     content NVARCHAR(MAX) NOT NULL,
24     is_resolved BIT NOT NULL DEFAULT 0,
25     created_at DATETIME2 NOT NULL DEFAULT GETDATE(),
26     updated_at DATETIME2 NULL,
27
28     CONSTRAINT fk_comments_review FOREIGN KEY (review_id)
29     REFERENCES reviews(review_id) ON DELETE CASCADE
30 );
31
32 CREATE TABLE approvals (
33     approval_id UNIQUEIDENTIFIER PRIMARY KEY DEFAULT NEWID(),
34     version_id UNIQUEIDENTIFIER NOT NULL,
35     user_id UNIQUEIDENTIFIER NOT NULL,
36     decision VARCHAR(20) NOT NULL,
37     rejection_reason NVARCHAR(MAX) NULL,
38     approved_at DATETIME2 NOT NULL DEFAULT GETDATE(),
39
40     CONSTRAINT chk_approvals_decision CHECK (decision IN ('
41     APPROVED', 'REJECTED'))
42 );
43
44 CREATE TABLE grounding_evidences (
45     evidence_id UNIQUEIDENTIFIER PRIMARY KEY DEFAULT NEWID(),
46     version_id UNIQUEIDENTIFIER NOT NULL,
47     snippet_content NVARCHAR(MAX) NOT NULL,
48     purpose VARCHAR(30) NOT NULL DEFAULT '
49     HALLUCINATION_PREVENTION',
50     created_at DATETIME2 NOT NULL DEFAULT GETDATE(),

```

```

47
48     CONSTRAINT chk_evidences_purpose CHECK (purpose IN ('
HALLUCINATION_PREVENTION', 'CODE_REFERENCE', '
AST_VERIFICATION'))
49 );
50
51 CREATE TABLE drift_alerts (
52     alert_id UNIQUEIDENTIFIER PRIMARY KEY DEFAULT NEWID(),
53     evidence_id UNIQUEIDENTIFIER NOT NULL,
54     repository_id UNIQUEIDENTIFIER NOT NULL,
55     drift_type VARCHAR(20) NOT NULL,
56     status VARCHAR(20) NOT NULL DEFAULT 'OPEN',
57     detected_at DATETIME2 NOT NULL DEFAULT GETDATE(),
58
59     CONSTRAINT fk_alerts_evidence FOREIGN KEY (evidence_id)
60     REFERENCES grounding_evidences(evidence_id) ON DELETE
CASCADE,
61     CONSTRAINT chk_alerts_type CHECK (drift_type IN ('
REFERENTIAL', 'SIGNATURE', 'SEMANTIC', 'OUTDATED')),
62     CONSTRAINT chk_alerts_status CHECK (status IN ('OPEN', '
RESOLVED', 'IGNORED'))
63 );
64
65
66
67 ALTER TABLE reviews ADD CONSTRAINT fk_reviews_version
68     FOREIGN KEY (version_id) REFERENCES document_versions(
version_id);
69 ALTER TABLE reviews ADD CONSTRAINT fk_reviews_user
70     FOREIGN KEY (user_id) REFERENCES users(user_id);
71 ALTER TABLE review_comments ADD CONSTRAINT fk_comments_user
72     FOREIGN KEY (user_id) REFERENCES users(user_id);
73 ALTER TABLE approvals ADD CONSTRAINT fk_approvals_version
74     FOREIGN KEY (version_id) REFERENCES document_versions(
version_id);
75 ALTER TABLE approvals ADD CONSTRAINT fk_approvals_user
76     FOREIGN KEY (user_id) REFERENCES users(user_id);
77 ALTER TABLE grounding_evidences ADD CONSTRAINT
fk_evidences_version
78     FOREIGN KEY (version_id) REFERENCES document_versions(
version_id);
79 ALTER TABLE drift_alerts ADD CONSTRAINT fk_alerts_repository
80     FOREIGN KEY (repository_id) REFERENCES repositories(
repository_id);
81
82 GO

```


Phần IV

Tổng Kết

Chương 6

Tổng Kết

Đề tài đã hoàn thành xuất sắc quy trình thiết kế cơ sở dữ liệu toàn diện cho hệ thống **LivingDocs** trải dài qua ba giai đoạn chuẩn hóa từ Stage 1 đến Stage 3. Ở Mức quan niệm (Stage 1), mô hình ERD và UML Class Diagram đã chuẩn hóa thành công các thực thể nghiệp vụ phức tạp như phân quyền Scoped RBAC, luồng kiểm duyệt 2 cấp và cơ chế lưu minh chứng AI. Chuyển sang Mức logic (Stage 2), hệ thống được ánh xạ chính xác sang Lược đồ quan hệ và chứng minh đạt chuẩn 3NF, giúp loại bỏ triệt để sự dư thừa dữ liệu cũng như các bất thường khi thêm, sửa, xóa. Tại Mức vật lý (Stage 3), các bảng được tối ưu hóa kiểu dữ liệu, hệ thống chỉ mục (Index) và ràng buộc toàn vẹn nhằm nâng cao hiệu năng truy vấn ở mức tối đa. Kết quả thu được tạo nên một nền tảng kiến trúc dữ liệu vững chắc, sẵn sàng đáp ứng khả năng mở rộng và vận hành ổn định cho dự án khi triển khai thực tế. Đây là minh chứng rõ nét cho tính hiệu quả của việc áp dụng quy trình thiết kế CSDL bài bản vào việc xây dựng một hệ thống phần mềm chuyên nghiệp.